Chapter A6
The Thumb Instruction Set

This chapter introduces the Thumb instruction set and describes how Thumb uses the ARM programmer's model. It contains the following sections:

- *About the Thumb instruction set* on page A6-2
- *Instruction set encoding* on page A6-4
- *Branch instructions* on page A6-6
- *Data-processing instructions* on page A6-8
- *Load and Store Register instructions* on page A6-15
- *Load and Store Multiple instructions* on page A6-18
- *Exception-generating instructions* on page A6-20
- *Undefined instruction space* on page A6-21.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.1 About the Thumb instruction set

The Thumb instruction set is a re-encoded subset of the ARM instruction set. Thumb is designed to increase the performance of ARM implementations that use a 16-bit or narrower memory data bus and to allow better code density than ARM. T variants of the ARM architecture incorporate both a full 32-bit ARM instruction set and the 16-bit Thumb instruction set. Every Thumb instruction is encoded in 16 bits.

Thumb does not alter the underlying programmer's model of the ARM architecture. It merely presents restricted access to it. All Thumb data-processing instructions operate on full 32-bit values, and full 32-bit addresses are produced by both data-access instructions and instruction fetches.

When the processor is executing Thumb instructions, eight general-purpose integer registers are available, R0 to R7, which are the same physical registers as R0 to R7 when executing ARM instructions. Some Thumb instructions also access the Program Counter (ARM Register 15), the Link Register (ARM Register 14) and the Stack Pointer (ARM Register 13). Further instructions allow limited access to ARM registers 8 to 15, which are know as the *high registers*.

When R15 is read, bit[0] is zero and bits[31:1] contain the PC. When R15 is written, bit[0] is IGNORED and bits[31:1] are written to the PC. Depending on how it is used, the value of the PC is either the address of the instruction plus 4 or is UNPREDICTABLE.

Thumb does not provide direct access to the CPSR or any SPSR (as the MSR and MRS instructions do in the ARM instruction set)). Thumb execution is flagged by the T bit (bit[5]) in the CPSR:

T == 0 32-bit instructions are fetched (and the PC is incremented by four) and are executed as ARM instructions.

T == 1 16-bit instructions are fetched (and the PC is incremented by two) and are executed as Thumb instructions.

Note

The Thumb instruction set is only compatible with the 32-bit ARM architectures. Thumb is not recommended for use with 26-bit architectures or with 26-bit compatibility options on 32-bit architectures.

6.1.1 Entering Thumb state

Thumb execution is normally entered by executing an ARM `BX` instruction (Branch and Exchange). This instruction branches to the address held in a general-purpose register, and if bit[0] of that register is 1, Thumb execution begins at the branch target address. If bit[0] of the target register is 0, ARM execution continues from the branch target address. On architecture versions 5 and above, `BLX` instructions and `LDR`/`LDM` instructions that load the PC can be used similarly.

Thumb execution can also be initiated by setting the T bit in the SPSR and executing an ARM instruction which restores the CPSR from the SPSR (a data-processing instruction with the S bit set and the PC as the destination, or a Load Multiple with Restore CPSR instruction). This allows an operating system to automatically restart a process independent of whether that process is executing Thumb code or ARM code.

The result is UNPREDICTABLE if the T bit is altered directly by writing the CPSR.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

6.1.2 Exceptions

Exceptions generated during Thumb execution switch to ARM execution before executing the exception handler (whose first instruction is at the hardware vector). The state of the T bit is preserved in the SPSR, and the LR of the exception mode is set so that the normal return instruction performs correctly, regardless of whether the exception occurred during ARM or Thumb execution. Table 6-1 lists the values of the exception mode LR for exceptions generated during Thumb execution.

Table 6-1 Exception return instructions

Exception	Exception link register value	Return instruction
Reset	UNPREDICTABLE value	-
Undefined	Address of undefined instruction + 2	`MOVS PC, R14`
SWI	Address of SWI instruction + 2	`MOVS PC, R14`
Prefetch Abort	Address of aborted instruction fetch + 4	`SUBS PC, R14, #4`
Data Abort	Address of the instruction that generated the abort + 8	`SUBS PC, R14, #8`
IRQ	Address of the next instruction to be executed + 4	`SUBS PC, R14, #4`
FIQ	Address of the next instruction to be executed + 4	`SUBS PC, R14, #4`

Note

For each exception, the return instruction indicated by Table 6-1 is the same as the return instruction required if the exception occurred during ARM execution, for the primary or only method of return from that instruction listed in *Exceptions* on page A2-13. However, the following two types of exception have a secondary return method, for which different return instructions are needed depending on whether the exception occurred during ARM or Thumb execution:

- For the Data Abort exception, the primary method of return causes execution to resume at the aborted instruction, which causes it to be re-executed. As described in *Data Abort (data access memory abort)* on page A2-17, it is also possible to return to the next instruction after the aborted instruction, using a `SUBS PC,R14,#4` instruction. If this type of return is required for a data abort caused by a Thumb instruction, use `SUBS PC,R14,#6` for the return instruction.
- For the Undefined Instruction exception, the primary method of return causes execution to resume at the next instruction after the undefined instruction. As described in *Undefined Instruction exception* on page A2-15, it is also possible to return to the undefined instruction itself, using the instruction `SUBS PC,R14,#4`. If this type of return is required for a Thumb undefined instruction, use `SUBS PC,R14,#2` for the return instruction. However, the main use of this type of return is for some types of coprocessor instruction, and as the Thumb instruction set does not contain any coprocessor instructions, you are unlikely to need this secondary method of return for Thumb instructions.

When these secondary methods of return are used, the exception handler code must test the SPSR T bit in order to determine which of the two return instructions to use.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.2 Instruction set encoding

Figure 6-1 shows the Thumb instruction set encoding. An entry in square brackets, for example [1], indicates a note on the following page.

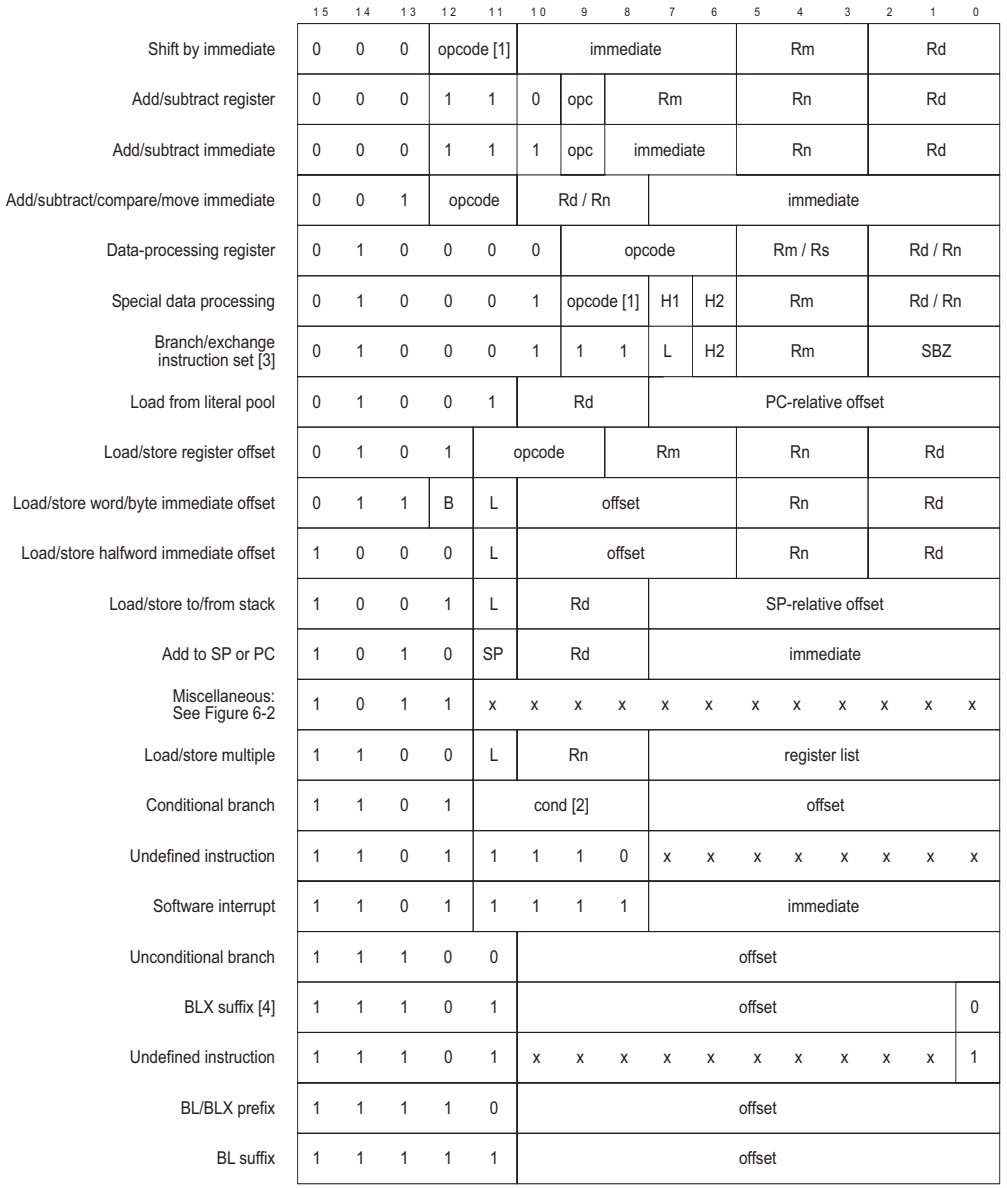


Figure 6-1 Thumb instruction set overview

Copyright © 1996-2000 ARM Limited. All rights reserved.

1. The opc field is not allowed to be 11 in this line. Other lines deal with the case that the opc field is 11.
2. The cond field is not allowed to be 1110 or 1111 in this line. Other lines deal with the cases where the cond field is 1110 or 1111.
3. The form with L==1 is UNPREDICTABLE prior to ARM architecture version 5.
4. This is an undefined instruction prior to ARM architecture version 5.

6.2.1 Miscellaneous instructions

Figure 6-2 lists miscellaneous Thumb instructions. An entry in square brackets, for example [1], indicates a note below the figure.

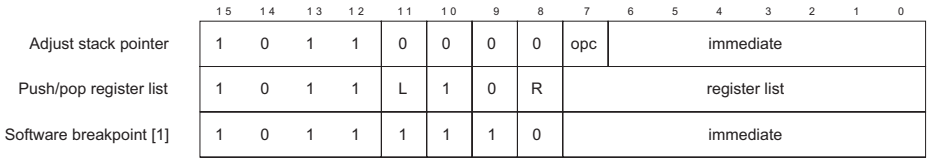


Figure 6-2 Miscellaneous Thumb instructions

1. This is an undefined instruction prior to ARM architecture version 5.

Note

Any instruction with bits[15:12] = 1011, and which is not shown in Figure 6-2, is an undefined instruction.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.3 Branch instructions

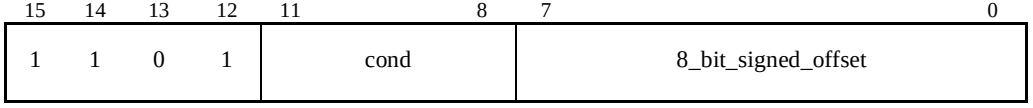
Thumb supports six types of branch instruction:

- an unconditional branch that allows a forward or backward branch of up to 2KB
- a conditional branch to allow forward and backward branches of up to 256 bytes
- a Branch with Link (subroutine call) is supported with a pair of instructions that allow forward and backward branches of up to 4MB
- a Branch and Exchange instruction branches to an address in a register and optionally switches to ARM code execution
- a Branch with Link and Exchange instruction performs a subroutine call to an address in a register and optionally switches to ARM code execution
- a second form of Branch with Link and Exchange uses a pair of instructions, similar to Branch with Link, but additionally switches to ARM code execution.

The encoding for these instructions is given below.

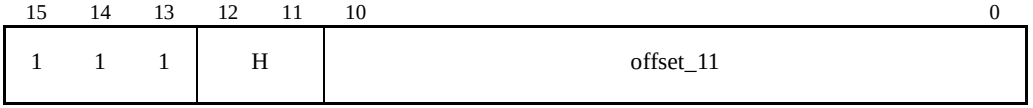
6.3.1 Conditional branch

```
B<cond>  <target_address>
```



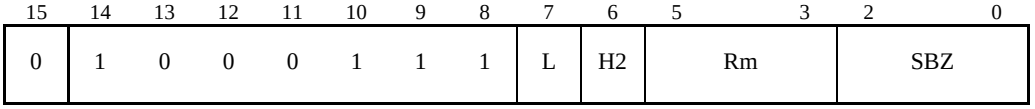
6.3.2 Unconditional branch

```
B    <target_address>
BL   <target_address>            ; Produces two 16-bit instructions
BLX  <target_address>            ; Produces two 16-bit instructions
```



6.3.3 Branch with exchange

```
BX    <Rm>
BLX   <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

6.3.4 Examples

```
    B    label        ; unconditionally branch to label

    BCC  label        ; branch to label if carry flag is clear

    BEQ  label        ; branch to label if zero flag is set

    BL   func         ; subroutine call to function

func
...                   ; Include body of function here
...
    MOV  PC, LR       ; R15=R14, return to instruction after the BL

    BX   R12          ; branch to address in R12; begin ARM execution if
                      ; bit 0 of R12 is zero; otherwise continue executing
                      ; Thumb code
```

6.3.5 List of branch instructions

The following instructions follow the formats shown above.

B — Conditional Branch. See *B (1)* on page A7-18.

B — Unconditional Branch. See *B (2)* on page A7-20.

BL — Branch with Link. See *BL, BLX(1)* on page A7-26.

BX — Branch and Exchange instruction set. See *BX* on page A7-32.

BLX — Branch with Link and Exchange instruction set. See *BL, BLX(1)* on page A7-26 and *BLX(2)* on page A7-30.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.4 Data-processing instructions

Thumb data-processing instructions are a subset of the ARM data-processing instructions, as shown in Table 6-2. All Thumb data-processing instructions in this table set the condition codes.

Table 6-2 Thumb data-processing instructions

Mnemonic	Operation	Action
`ADC Rd, Rm`	Add with Carry	Rd := Rd + Rm + Carry flag
`ADD Rd, Rn, Rm`	Add	Rd := Rn + Rm
`ADD Rd, Rn, #0 to 7`	Add	Rd := Rn + 3-bit immediate
`ADD Rd, #0 to 255`	Add	Rd := Rd + 8-bit immediate
`AND Rd, Rm`	Logical AND	Rd := Rd AND Rm
`ASR Rd, Rm, #1 to 32`	Arithmetic Shift Right	Rd := Rm ASR 5-bit immediate
`ASR Rd, Rs`	Arithmetic Shift Right	Rd := Rd ASR Rs
`BIC Rd, Rm`	Bit Clear	Rd := Rd AND NOT Rm
`CMN Rn, Rm`	Compare Negated	Update flags after Rn + Rm
`CMP Rn, #0 to 255`	Compare	Update flags after Rn - 8-bit immediate
`CMP Rn, Rm`	Compare	Update flags after Rn - Rm
`EOR Rd, Rm`	Logical Exclusive OR	Rd := Rd EOR Rm
`LSL Rd, Rm, #0 to 31`	Logical Shift Left	Rd := Rm LSL 5-bit immediate
`LSL Rd, Rs`	Logical Shift Left	Rd := Rd LSL Rs
`LSR Rd, Rm, #1 to 32`	Logical Shift Right	Rd := Rm LSR 5-bit immediate
`LSR Rd, Rs`	Logical Shift Right	Rd := Rd LSR Rs
`MOV Rd, #0 to 255`	Move	Rd := 8-bit immediate
`MOV Rd, Rn`	Move	Rd := Rn
`MUL Rd, Rm`	Multiply	Rd := Rm x Rd
`MVN Rd, Rm`	Move Not	Rd := NOT Rm
`NEG Rd, Rm`	Negate	Rd := 0 - Rm
`ORR Rd, Rm`	Logical (inclusive) OR	Rd := Rd OR Rm

Copyright © 1996-2000 ARM Limited. All rights reserved.

Table 6-2 Thumb data-processing instructions (Continued)

Mnemonic	Operation	Action
`ROR Rd, Rs`	Rotate Right	Rd := Rd ROR Rs
`SBC Rd, Rm`	Subtract with Carry	Rd := Rd - Rm - NOT(Carry Flag)
`SUB Rd, Rn, Rm`	Subtract	Rd := Rn - Rm
`SUB Rd, Rn, #0 to 7`	Subtract	Rd := Rn - 3-bit immediate
`SUB Rd, #0 to 255`	Subtract	Rd := Rd - 8-bit immediate
`TST Rn, Rm`	Test	Update flags after Rn AND Rm

For example:

```
ADD    R0, R4, R7        ; R0 = R4 + R7
SUB    R6, R1, R2        ; R6 = R1 - R2
ADD    R0, #255          ; R0 = R0 + 255
ADD    R1, R4, #4        ; R1 = R4 + 4
NEG    R3, R1            ; R3 = 0 - R1
AND    R2, R5            ; R2 = R2 AND R5
EOR    R1, R6            ; R1 = R1 EOR R6
CMP    R2, R3            ; update flags after R2 - R3
CMP    R7, #100          ; update flags after R7 - 100
MOV    R0, #200          ; R0 = 200
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

6.4.1 High registers

There are seven types of data-processing instruction which operate on ARM registers 8 to 14 and the PC as shown in Table 6-3. Apart from `CMP`, instructions in this table do not change the condition code flags.

Table 6-3 High register data-processing instructions

Mnemonic	Operation	Action
`MOV Rd, Rn`	Move	Rd := Rn
`ADD Rd, Rm`	Add	Rd := Rd + Rm
`CMP Rn, Rm`	Compare	Update flags after Rn - Rm
`ADD SP, #0 to 508`	Increment stack pointer	R13 = R13 + 4* (7-bit immediate)
`SUB SP, #0 to 508`	Decrement stack pointer	R13 = R13 - 4* (7-bit immediate)
`ADD Rd, SP, #0 to 1020`	Form Stack address	Rd = R13 + 4* (8-bit immediate)
`ADD Rd, PC, #0 to 1020`	Form PC address	Rd = PC + 4* (8-bit immediate)

For example:

```
MOV    R0, R12              ; R0 = R12
ADD    R10, R1, R2          ; R6 = R1 - R2
MOV    PC, LR               ; PC = R14
CMP    R10, R11             ; update flags after R10 - R11
SUB    SP, #12              ; increase stack size by 12 bytes
ADD    SP, #16              ; decrease stack size by 16 bytes
ADD    R2, SP, #20          ; R2 = SP + 20
ADD    R0, PC, #500         ; R0 = PC + 500
```

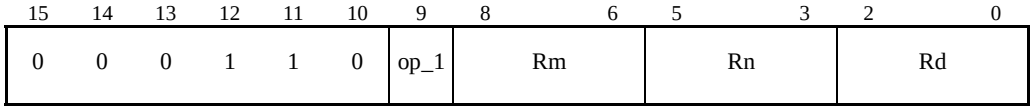

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.4.2 Formats

Data-processing instructions use the following eight instruction formats:

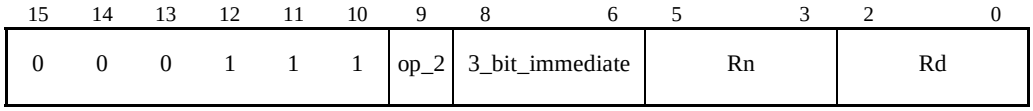
Format 1

```
<opcode1>  <Rd>, <Rn>, <Rm>
<opcode1> := ADD | SUB
```



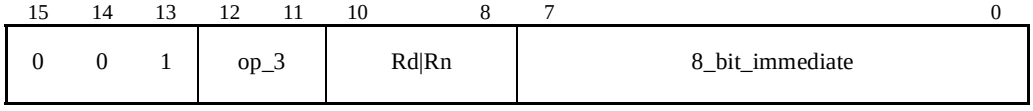
Format 2

```
<opcode2>  <Rd>, <Rn>, #<3_bit_immed>
<opcode2> := ADD | SUB
```



Format 3

```
<opcode3>  <Rd>|<Rn>, #<8_bit_immed>
<opcode3> := ADD | SUB | MOV | CMP
```



Format 4

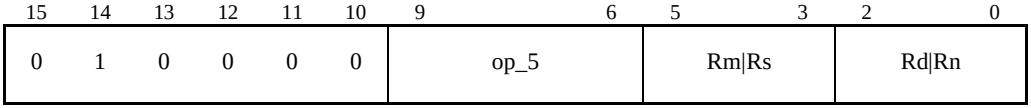
```
<opcode4>  <Rd>, <Rm>, #<shift_imm>
<opcode4> := LSL | LSR | ASR
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

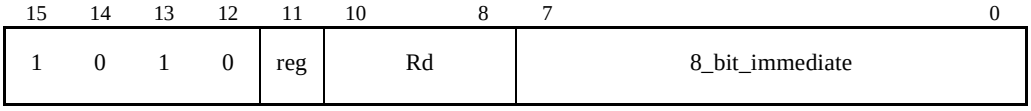
Format 5

```
<opcode5>  <Rd>|<Rn>, <Rm>|<Rs>
<opcode5> := MVN | CMP | CMN | TST | ADC | SBC | NEG | MUL |
             LSL | LSR | ASR | ROR | AND | EOR | ORR | BIC
```



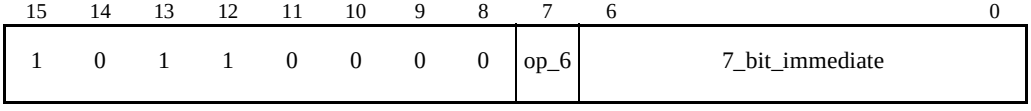
Format 6

```
ADD  <Rd>, <reg>, #<8_bit_immed>
<reg> := SP | PC
```



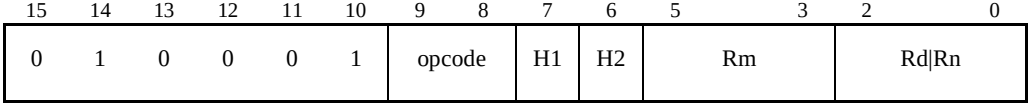
Format 7

```
<opcode6>  SP, SP, #<7_bit_immed>
<opcode6> := ADD | SUB
```



Format 8

```
<opcode7>  <Rd>|<Rn>, <Rm>
<opcode7> := MOV | ADD | CMP
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

6.4.3 List of data-processing instructions

The following instructions follow the formats shown above.

`ADC` Add with Carry. See *ADC* on page A7-4.

`ADD` Add (immediate). See *ADD (1)* on page A7-5.

`ADD` Add (large immediate). See *ADD (2)* on page A7-6.

`ADD` Add (register). See *ADD (3)* on page A7-7.

`ADD` Add high registers. See *ADD (4)* on page A7-8.

`ADD` Add (immediate to program counter). See *ADD (5)* on page A7-10.

`ADD` Add (immediate to stack pointer). See *ADD (6)* on page A7-11.

`ADD` Increment stack pointer. See *ADD (7)* on page A7-12.

`AND` Logical AND. See *AND* on page A7-13.

`ASR` Arithmetic Shift Right (immediate). See *ASR (1)* on page A7-14.

`ASR` Arithmetic Shift Right (register). See *ASR (2)* on page A7-16.

`BIC` Bit Clear. See *BIC* on page A7-22.

`CMN` Compare Negative (register). See *CMN* on page A7-34.

`CMP` Compare (immediate). See *CMP (1)* on page A7-35.

`CMP` Compare (register). See *CMP (2)* on page A7-36.

`CMP` Compare high registers. See *CMP (3)* on page A7-37.

`EOR` Exclusive OR. See *EOR* on page A7-39.

`LSL` Logical Shift Left (immediate). See *LSL (1)* on page A7-59.

`LSL` Logical Shift Left (register). See *LSL (2)* on page A7-60.

`LSR` Logical Shift Right (immediate). See *LSR (1)* on page A7-62.

`LSR` Logical Shift Right (register). See *LSR (2)* on page A7-64.

`MOV` Move (immediate). See *MOV (1)* on page A7-66.

`MOV` Move a low register to another low register. See *MOV (2)* on page A7-67.

`MOV` Move high registers. See *MOV (3)* on page A7-68.

`MUL` Multiply. See *MUL* on page A7-70.

`MVN` Move NOT (register). See *MVN* on page A7-72.

`NEG` Negate (register). See *NEG* on page A7-73.

Copyright © 1996-2000 ARM Limited. All rights reserved.

ORR	Logical OR. See *ORR* on page A7-74.
ROR	Rotate Right (register). See *ROR* on page A7-80.
SBC	Subtract with Carry (register). See *SBC* on page A7-82.
SUB	Subtract (immediate). See *SUB (1)* on page A7-98.
SUB	Subtract (large immediate). See *SUB (2)* on page A7-99.
SUB	Subtract (register). See *SUB (3)* on page A7-100.
SUB	Decrement stack pointer. See *SUB (4)* on page A7-101.
TST	Test (register). See *TST* on page A7-103.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.5 Load and Store Register instructions

Thumb supports eight types of Load and Store Register instructions. Two basic addressing modes are available. These allow the load and store of words, halfwords and bytes, and also the load of signed halfwords and bytes:

- register plus register
- register plus 5-bit immediate (not available for signed halfword and signed byte loads).

If an immediate offset is used, it is scaled by 4 for word access and 2 for halfword accesses.

In addition, three special instructions allow:

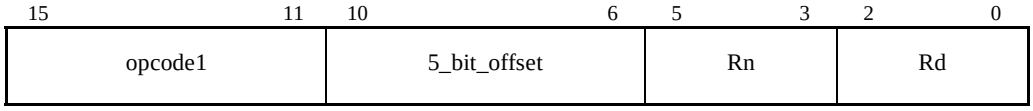
- words to be loaded using the PC as a base with a 1KB (word-aligned) immediate offset
- words to be loaded and stored with the stack pointer (R13) as the base and a 1KB (word-aligned) immediate offset.

6.5.1 Formats

Load and Store Register instructions have the following formats:

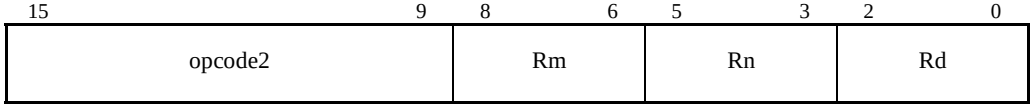
Format 1

```
<opcode1>  <Rd>, [<Rn>, #<5_bit_offset>]
<opcode1> := LDR|LDRH|LDRB|STR|STRH|STRB
```



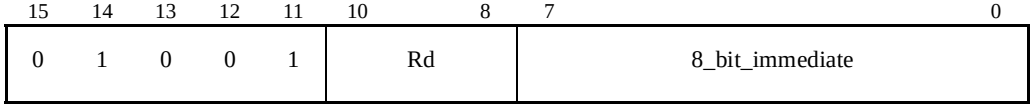
Format 2

```
<opcode2>  <Rd>, [<Rn>, <Rm>]
<opcode2> := LDR|LDRH|LDRSH|LDRB|LDRSB|STR|STRH|STRB
```



Format 3

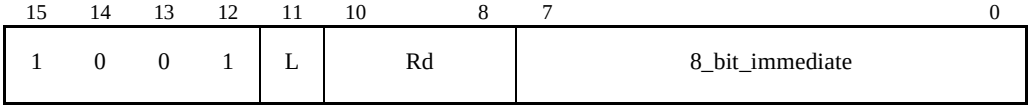
```
LDR  <Rd>, [PC, #<8_bit_offset>]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

Format 4

```
<opcode3>  <Rd>, [SP, #<8_bit_offset>]
<opcode3>  := LDR | STR
```



For example:

```
LDR    R4, [R2, #4]           ; Load word into R4 from address R2 + 4

LDR    R4, [R2, R1]           ; Load word into R4 from address R2 + R1

STR    R0, [R7, #0x7C]        ; Store word from R0 to address R7 + 124

STRB   R1, [R5, #31]          ; Store byte from R1 to address R5 + 31

STRH   R4, [R2, R3]           ; Store halfword from R4 to R2 + R3

LDRH   R3, [R6, R5]           ; Load word into R3 from R6 + R5

LDRB   R2, [R1, #5]           ; Load byte into R2 from R1 + 5

LDR    R6, [PC, #0x3FC]       ; Load R6 from PC + 0x3FC

LDR    R5, [SP, #64]          ; Load R5 from SP + 64

STR    R4, [SP, #0x260]       ; Load R5 from SP + 0x260
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

6.5.2 List of Load and Store Register instructions

The following instructions follow the formats shown above.

LDR	Load Word (immediate offset). See *LDR (1)* on page A7-42.
LDR	Load Word (register offset). See *LDR (2)* on page A7-44.
LDR	Load Word (PC-relative). See *LDR (3)* on page A7-46.
LDR	Load Word (SP-relative). See *LDR (4)* on page A7-48.
LDRB	Load Unsigned Byte (immediate offset). See *LDRB (1)* on page A7-50.
LDRB	Load Unsigned Byte (register offset). See *LDRB (2)* on page A7-51.
LDRH	Load Unsigned Halfword (immediate offset). See *LDRH (1)* on page A7-52.
LDRH	Load Unsigned Halfword (register offset). See *LDRH (2)* on page A7-54.
LDRSB	Load Signed Byte (register offset). See *LDRSB* on page A7-56.
LDRSH	Load Signed Halfword (register offset). See *LDRSH* on page A7-57.
STR	Store Word (immediate offset). See *STR (1)* on page A7-86.
STR	Store Word (register offset). See *STR (2)* on page A7-88.
STR	Store Word (SP-relative). See *STR (3)* on page A7-90.
STRB	Store Byte (immediate offset). See *STRB (1)* on page A7-92.
STRB	Store Byte (register offset). See *STRB (2)* on page A7-93.
STRH	Store Halfword (immediate offset). See *STRH (1)* on page A7-94.
STRH	Store Halfword (register offset). See *STRH (2)* on page A7-96.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.6 Load and Store Multiple instructions

Thumb supports four types of Load and Store Multiple instructions:

- Two instructions, LDMIA and STMIA, are designed to support block copy. They have a fixed Increment After addressing mode from a base register.
- The other two instructions, PUSH and POP, also have a fixed addressing mode. They implement a full descending stack and the stack pointer (R13) is used as the base register.

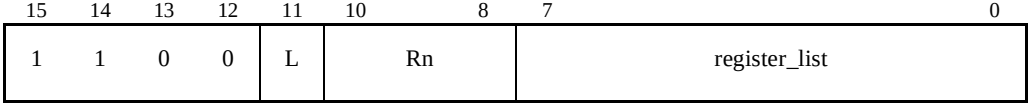
All four instructions update the base register after transfer and all can transfer any or all of the lower 8 registers. PUSH can also stack the return address, and POP can load the PC.

6.6.1 Formats

Load and Store Multiple instructions have the following formats:

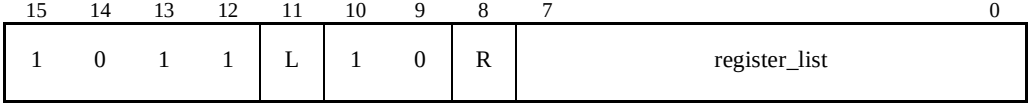
Format 1

```
<opcode1>  <Rn>!, <registers>
<opcode1>  := LDMIA | STMIA
```



Format 2

```
PUSH  {<registers>}
POP   {<registers>}
```



6.6.2 Examples

```
    LDMIA    R7!, {R0-R3, R5}         ; Load R0 to R3-R5 from R7, add 20 to R7
    STMIA    R0!, {R3, R4, R5}        ; Store R3-R5 to R0: add 12 to R0

function
    PUSH     {R0-R7, LR}              ; push onto the stack (R13) R0-R7 and
                                      ; the return address
...                                   ; code of the function body
...
    POP      {R0-R7, PC}              ; restore R0-R7 from the stack
                                      ; and the program counter, and return
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

6.6.3 List of Load and Store Multiple instructions

The following instructions follow the formats shown above.

`LDMIA` Load Multiple. See *LDMIA* on page A7-40.

`POP` Pop Multiple. See *POP* on page A7-75.

`PUSH` Push Multiple. See *PUSH* on page A7-78.

`STMIA` Store Multiple. See *STMIA* on page A7-84.

Copyright © 1996-2000 ARM Limited. All rights reserved.

6.7 Exception-generating instructions

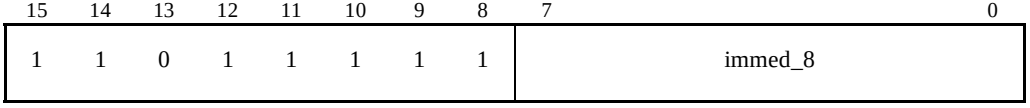
The Thumb instruction set provides two types of instruction whose main purpose is to cause a processor exception to occur:

- The Software Interrupt (`SWI`) instruction is used to cause a SWI exception to occur (see *Software Interrupt exception* on page A2-16). This is the main mechanism in the Thumb instruction set by which User mode code can make calls to privileged Operating System code.
- The Breakpoint (`BKPT`) instruction is used for software breakpoints in T variants of ARM architecture versions 5 and above. Its default behavior is to cause a Prefetch Abort exception to occur (see *Prefetch Abort (instruction fetch memory abort)* on page A2-16). A debug monitor program that has previously been installed on the Prefetch Abort vector can handle this exception.

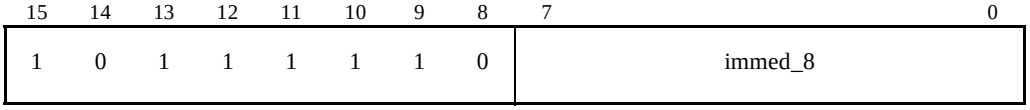
  If debug hardware is present in the system, it is allowed to override this default behavior. Details of whether and how this happens are IMPLEMENTATION DEFINED.

6.7.1 Instruction encodings

`SWI  <immed_8>`



`BKPT  <immed_8>`



In both `SWI` and `BKPT`, the immed_8 field of the instruction is ignored by the ARM processor. The SWI or Prefetch Abort handler can optionally be written to load the instruction that caused the exception and extract these fields. This allows them to be used to communicate extra information about the Operating System call or breakpoint to the handler.

6.7.2 List of exception-generating instructions

`BKPT` Breakpoint. See *BKPT* on page A7-24.

`SWI` Software Interrupt. See *SWI* on page A7-102.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

6.8 Undefined instruction space

The following instructions are UNDEFINED in the Thumb instruction set:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	1	x	x	x	x	x	x	x	x
1	0	1	1	0	x	1	x	x	x	x	x	x	x	x	x
1	0	1	1	1	0	x	x	x	x	x	x	x	x	x	x
1	0	1	1	1	1	1	1	x	x	x	x	x	x	x	x
1	1	0	1	1	1	1	0	x	x	x	x	x	x	x	x

In general, these instructions can be used to extend the Thumb instruction set in the future. However, it is intended that the following group of instructions will not be used in this manner:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	1	1	0	x	x	x	x	x	x	x	x

Use one of these instructions if you want to use an undefined instruction for software purposes, with minimal risk that future hardware will treat it as a defined instruction.

Copyright © 1996-2000 ARM Limited. All rights reserved.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Chapter A7
Thumb Instructions

This chapter describes the syntax and usage of every Thumb instruction, in the section:

- *Alphabetical list of Thumb instructions* on page A7-2
- *Thumb instructions and architecture versions* on page A7-104.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1 Alphabetical list of Thumb instructions

Every Thumb instruction is listed on the following pages. Each instruction description shows:

- the instruction encoding
- the instruction syntax
- the versions of the ARM architecture where the instruction is valid
- any exceptions that might apply
- a pseudo-code specification of how the instruction operates
- notes on usage and special cases
- the equivalent ARM instruction encoding.

7.1.1 General notes

These notes explain the types of information and abbreviations used on the instruction pages.

Syntax abbreviations

The following abbreviations are used in the instruction pages:

`immed_<n>` This is an `<n>`-bit immediate value. For example, an 8-bit immediate value is represented by:

```
immed_8
```

`signed_immed_<n>`

This is a signed immediate. For example, an 8-bit signed immediate is represented by:

```
signed_immed_8
```

Architecture version

For the convenience of the reader, this section describes the version of the ARM instruction set that the instruction is associated with, not the version of the Thumb instruction set. There have been two versions of the Thumb instruction set architecture to date:

THUMBv1 This is used in T variants of version 4 of the ARM instruction set architecture.

THUMBv2 This is used in T variants of version 5 and above of the ARM instruction set architecture.

Instructions which are described as being in all T variants are therefore present in both THUMBv1 and THUMBv2, while those that are described as being in T variants of version 5 and above are in THUMBv2 only.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

This section shows the syntax and encoding of an equivalent ARM instruction. When no precise equivalent is available, a close equivalent is shown and the reasons why it is not a precise equivalent are explained.

A common reason for the instruction not being a precise equivalent is that it reads the value of the PC. This produces the instruction's own address plus N, where N is 8 for ARM instructions and 4 for Thumb instructions. This difference can often be compensated for by adjusting an immediate constant in the equivalent ARM instruction.

In the equivalent instruction encodings, named fields and bits must be filled in with the corresponding fields and bits from the Thumb instruction, or in a few cases with values derived from the Thumb instruction as described in the text.

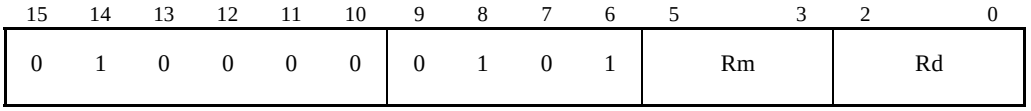
The ARM instruction fields are normally the same length as the corresponding Thumb instruction fields, with one important exception. Thumb register fields are normally 3 bits long, whereas ARM register fields are normally 4 bits long. In these cases, the Thumb register field must be extended with a high-order 0 when substituted into the ARM register field, so that the ARM instruction refers to the correct one of R0 to R7.

Information on usage

Usage information is only given for Thumb instructions where it differs significantly from ARM instruction usage. If no Usage section appears for a Thumb instruction, see the equivalent ARM instruction page in Chapter A4 *ARM Instructions* for usage information.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.2 ADC



The ADC (Add with Carry) instruction can be used to synthesize multi-word addition. The condition code flags are updated, based on the result.

Syntax

```
ADC  <Rd>, <Rm>
```

where:

`<Rd>` Holds the first value for the addition, and is the destination register for the operation.

`<Rm>` Specifies the register that contains the second operand for the addition.

Architecture version

All T variants

Exceptions

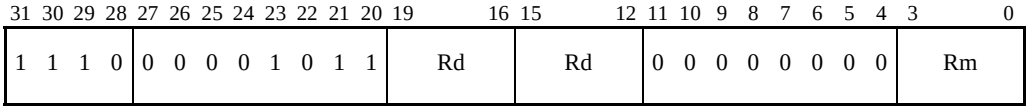
None

Operation

```
Rd = Rd + Rm + C Flag
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = CarryFrom(Rd + Rm + C Flag)
V Flag = OverflowFrom(Rd + Rm + C Flag)
```

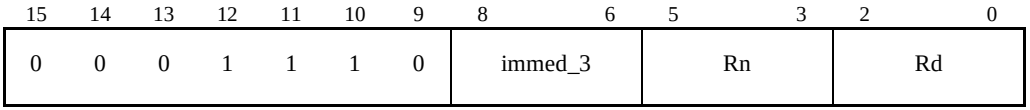
Equivalent ARM syntax and encoding

```
ADCS  <Rd>, <Rd>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.3 ADD (1)



This form of ADD adds a small constant value to the value of a register and stores the result in a second register. The condition code flags are updated, based on the result.

Syntax

```
ADD  <Rd>, <Rn>, #<immed_3>
```

where:

`<Rd>` Is the destination register for the completed operation.

`<Rn>` Specifies the register that contains the operand for the addition.

`<immed_3>` Specifies a 3-bit immediate value that is added to the value of `<Rn>`.

Architecture version

All T variants

Exceptions

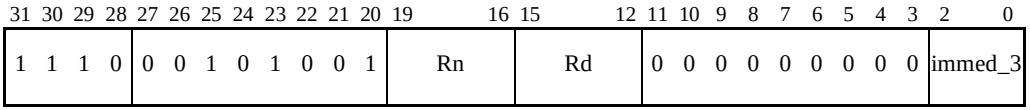
None

Operation

```
Rd = Rn + immed_3
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = CarryFrom(Rn + immed_3)
V Flag = OverflowFrom(Rn + immed_3)
```

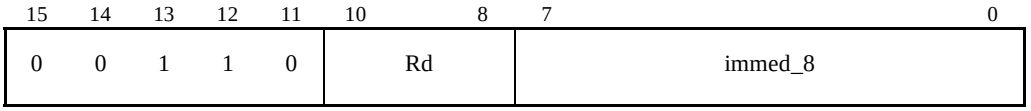
Equivalent ARM syntax and encoding

```
ADDS  <Rd>, <Rn>, #<immed_3>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.4 ADD (2)



This form of ADD adds a large immediate value to the value of a register and stores the result back in the same register. The condition code flags are updated, based on the result.

Syntax

```
ADD  <Rd>, #<immed_8>
```

where:

`<Rd>` Holds the first operand for the addition, and is the destination register for the completed operation.

`<immed_8>` Specifies an 8-bit immediate value that is added to the value of `<Rd>`.

Architecture version

All T variants

Exceptions

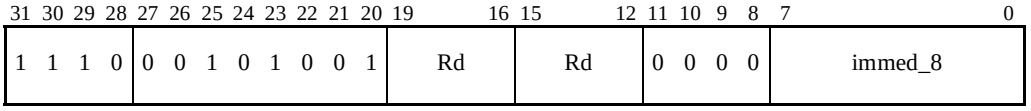
None

Operation

```
Rd = Rd + immed_8
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = CarryFrom(Rd + immed_8)
V Flag = OverflowFrom(Rd + immed_8)
```

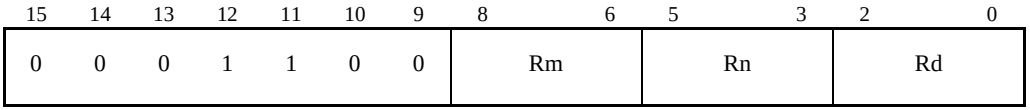
Equivalent ARM syntax and encoding

```
ADDS  <Rd>, <Rd>, #<immed_8>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.5 ADD (3)



This form of ADD adds the value of one register to the value of a second register, and stores the result in a third register. The condition code flags are updated, based on the result.

Syntax

```
ADD  <Rd>, <Rn>, <Rm>
```

where:

`<Rd>` Is the destination register for the completed operation.

`<Rn>` Specifies the register containing the first value for the addition.

`<Rm>` Specifies the register containing the second value for the addition.

Architecture version

All T variants

Exceptions

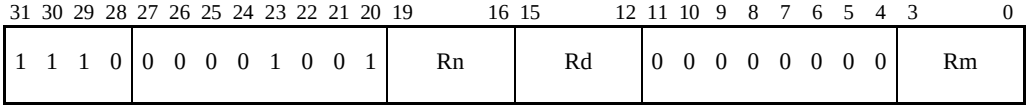
None

Operation

```
Rd = Rn + Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = CarryFrom(Rn + Rm)
V Flag = OverflowFrom(Rn + Rm)
```

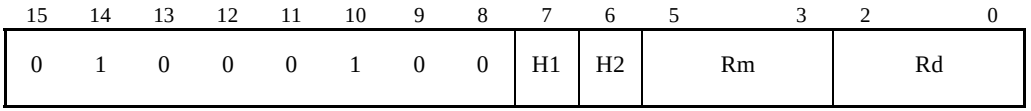
Equivalent ARM syntax and encoding

```
ADDS  <Rd>, <Rn>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.6 ADD (4)



This form of ADD adds the values of two registers, one or both of which are high registers. Unlike the low-register ADD instruction (*ADD (3)* on page A7-7), this instruction does not change the flags.

Syntax

```
ADD  <Rd>, <Rm>
```

where:

`<Rd>` Specifies the register containing the first value, and is also the destination register. It can be any of R0 to R15. The register number is encoded in the instruction in H1 (most significant bit) and Rd (remaining three bits).

`<Rm>` Specifies the register containing the second value. It can be any of R0 to R15. Its number is encoded in the instruction in H2 (most significant bit) and Rm (remaining three bits).

Architecture version

All T variants

Exceptions

None

Operation

```
Rd = Rd + Rm
```

Notes

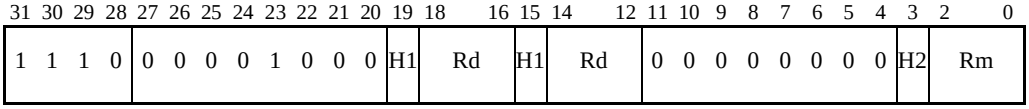
Operand restriction If a low register is specified for `<Rd>` and Rm (H1==0 and H2==0), the result is UNPREDICTABLE.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

A close equivalent is:

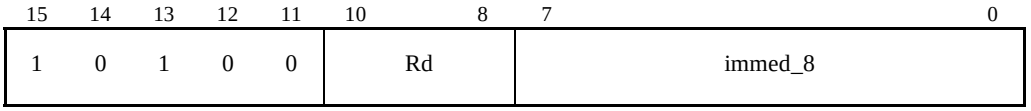
```
ADD  <Rd>, <Rd>, <Rm>
```



There are slight differences when the instruction accesses the PC, because of the different definitions of the PC when executing ARM and Thumb code.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.7 ADD (5)



This form of ADD adds an immediate value to the PC and writes the resulting PC-relative address to a destination register. The immediate can be any multiple of 4 in the range 0 to 1020. The condition codes are not affected.

Syntax

```
ADD  <Rd>, PC, #<immed_8> * 4
```

where:

`<Rd>` Is the destination register for the completed operation.

`PC` Indicates PC-relative addressing.

`<immed_8>` Specifies an 8-bit immediate value that is quadrupled and added to the value of the PC.

Architecture version

All T variants

Exceptions

None

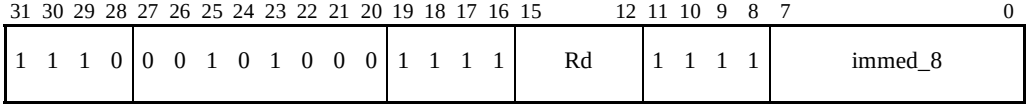
Operation

```
Rd = (PC AND 0xFFFFFFFC) + (immed_8 << 2)
```

Equivalent ARM syntax and encoding

A close equivalent is:

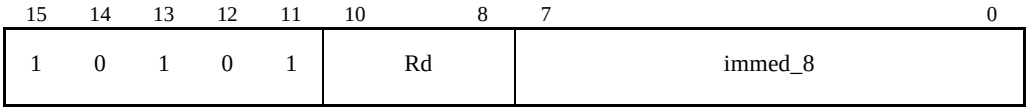
```
ADD  <Rd>, PC, #<immed_8> * 4
```



The definitions of the PC differ between ARM and Thumb code. This makes a difference between the precise results of the instructions.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.8 ADD (6)



This form of ADD adds an immediate value to the SP and writes the resulting SP-relative address to a destination register. The immediate can be any multiple of 4 in the range 0 to 1020. The condition codes are not affected.

Syntax

```
ADD  <Rd>, SP, #<immed_8> * 4
```

where:

`<Rd>` Is the destination register for the completed operation.

`SP` Indicates SP-relative addressing.

`<immed_8>` Specifies an 8-bit immediate value that is quadrupled and added to the value of the SP.

Architecture version

All T variants

Exceptions

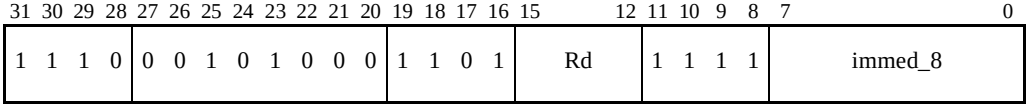
None

Operation

```
Rd = SP + (immed_8 << 2)
```

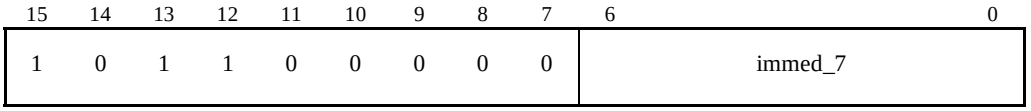
Equivalent ARM syntax and encoding

```
ADD  <Rd>, SP, #<immed_8> * 4
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.9 ADD (7)



This form of ADD increments the SP by four times a 7-bit immediate (that is, by a multiple of 4 in the range 0 to 508). The condition codes are not affected.

Syntax

```
ADD  SP, #<immed_7> * 4
```

where:

SP Contains the first operand for the addition. SP is also the destination register for the operation.

<immed_7> Specifies the immediate value that is quadrupled and added to the value of the SP.

Architecture version

All T variants

Exceptions

None

Operation

```
SP = SP + (immed_7 << 2)
```

Usage

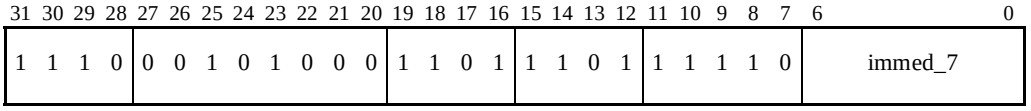
For the Full Descending stack which the Thumb instruction set is designed to use, incrementing the SP is used to discard data on the top of the stack.

Notes

Alternative syntax This instruction can also be written as `ADD SP, SP, #(<immed_7> * 4)`.

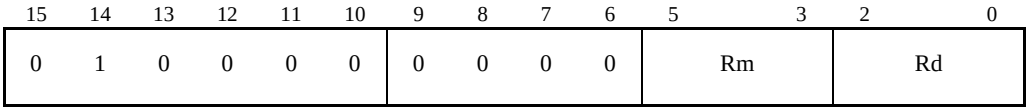
Equivalent ARM syntax and encoding

```
ADD  SP, SP, #<immed_7> * 4
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.10 AND



The AND (Logical AND) instruction performs a bitwise AND of the values in two registers. The condition code flags are updated, based on the result.

Syntax

```
AND  <Rd>, <Rm>
```

where:

<Rd> Specifies the register containing the first operand, and is also the destination register.

<Rm> Specifies the register containing the second operand.

Architecture version

All T variants

Exceptions

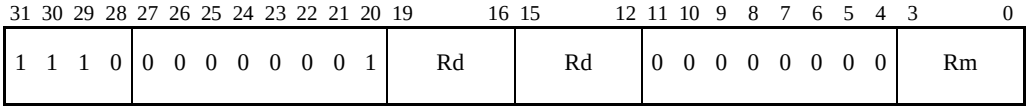
None

Operation

```
Rd = Rd AND Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

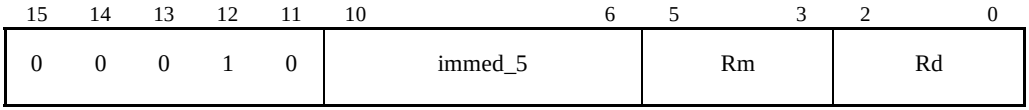
Equivalent ARM syntax and encoding

```
ANDS  <Rd>, <Rd>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.11 ASR (1)



This form of the ASR (Arithmetic Shift Right) instruction is used to provide the signed value of a register divided by a constant power of 2. The condition code flags are updated, based on the result.

Syntax

```
ASR  <Rd>, <Rm>, #<immed_5>
```

where:

`<Rd>` Is the destination register for the completed operation.

`<Rm>` Specifies the register that contains the value to be shifted.

`<immed_5>` Specifies the shift amount, in the range 1 to 32. Shifts by 1 to 31 are encoded directly in immed_5. A shift by 32 is encoded as immed_5 == 0.

Architecture version

All T variants

Exceptions

None

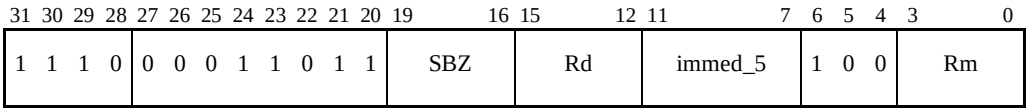
Operation

```
if immed_5 == 0
    C Flag = Rm[31]
    if Rm[31] == 0 then
        Rd = 0
    else /* Rm[31] == 1 */]
        Rd = 0xFFFFFFFF
else /* immed_5 > 0 */
    C Flag = Rm[immed_5 - 1]
    Rd = Rm Arithmetic_Shift_Right immed_5
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

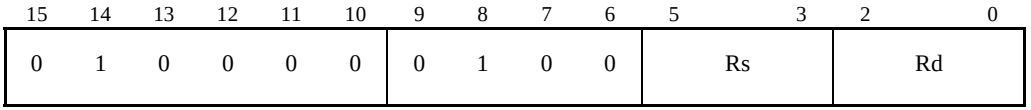
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rm>, ASR #<immed_5>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.12 ASR (2)



This form of ASR is used to provide the signed value of a register divided by a variable power of 2. The condition code flags are updated, based on the result.

Syntax

```
ASR  <Rd>, <Rs>
```

where:

`<Rd>` Contains the value to be shifted, and is also the destination register for the completed operation.

`<Rs>` Specifies the register that contains the value of the shift.

Architecture version

All T variants

Exceptions

None

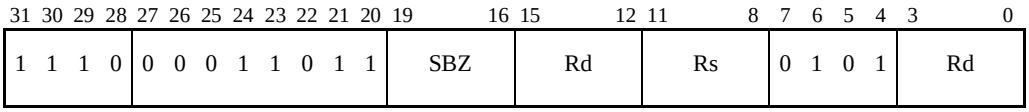
Operation

```
if Rs[7:0] == 0 then
    C Flag = unaffected
    Rd = unaffected
else if Rs[7:0] < 32 then
    C Flag = Rd[Rs[7:0] - 1]
    Rd = Rd Arithmetic_Shift_Right Rs[7:0]
else /* Rs[7:0] >= 32 */
    C Flag = Rd[31]
    if Rd[31] == 0 then
        Rd = 0
    else /* Rd[31] == 1 */
        Rd = 0xFFFFFFFF
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

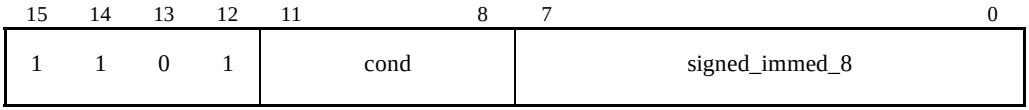
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rd>, ASR <Rs>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.13 B (1)



This form of the B (Branch) instruction provides a conditional branch to a target address.

Syntax

```
B<cond>  <target_address>
```

where:

`<cond>` Is the condition under which the instruction is executed. The conditions are defined in *The condition field* on page A3-5.

`<target_address>`

Specifies the address to branch to. The branch target address is calculated by:

1. Shifting the 8-bit signed offset field of the instruction left by one bit.
2. Sign-extending the result to 32 bits.
3. Adding this to the contents of the PC (which contains the address of the branch instruction plus 4).

The instruction can therefore specify a branch of approximately ±256 bytes.

Architecture version

All T variants

Exceptions

None

Operation

```
if ConditionPassed(cond) then
    PC = PC + (SignExtend(signed_immed_8) << 1)
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Usage

To calculate the correct value of signed_immed_8, the assembler (or other toolkit component) needs to:

1. Form the base address for the branch. This is the address of the branch instruction, plus 4. In other words, the base address is equal to the PC value read by that instruction.
2. Subtract the base address from the target address to form a byte offset. This offset is always even, because all Thumb instructions are halfword-aligned.
3. If the byte offset is outside the range -256 to +254, use an alternative code-generation strategy or produce an error as appropriate.
4. Otherwise, set the signed_immed_8 field of the instruction to the byte offset divided by 2.

Notes

Memory bounds Branching backwards past location zero and forwards over the end of the 32-bit address space is UNPREDICTABLE.

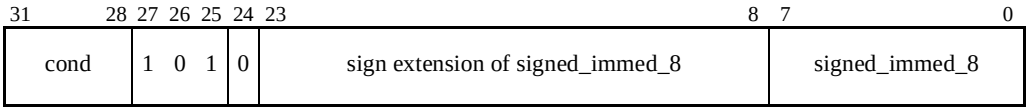
AL condition If the condition field indicates `AL` (0b1110), the instruction is instead UNDEFINED. When an unconditional branch is required, use the unconditional Branch instruction described in *B (2)* on page A7-20.

NV condition If the condition field indicates `NV` (0b1111), the instruction is a `SWI` instead (see *SWI* on page A7-102).

Equivalent ARM syntax and encoding

A close equivalent is:

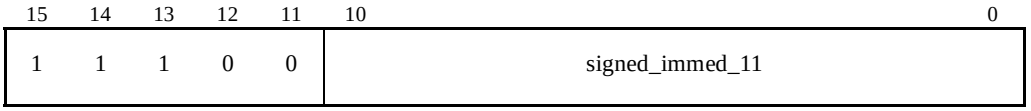
```
B<cond>  <target_address>
```



This differs from the Thumb instruction, because the offset in the ARM instruction is shifted left by 2 before being added to the PC, whereas the offset in the Thumb instruction is shifted left by 1. Also, the PC values read by the ARM and Thumb instructions are different.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.14 B (2)



This form of B provides an unconditional branch to a target address.

Syntax

```
B  <target_address>
```

where:

`<target_address>`

Specifies the address to branch to. The branch target address is calculated by:

1. Shifting the 11-bit signed offset of the instruction left one bit.
2. Sign-extending the result to 32 bits.
3. Adding this to the contents of the PC (which contains the address of the branch instruction plus 4).

The instruction can therefore specify a branch of approximately ±2048 bytes.

Architecture version

All T variants

Exceptions

None

Operation

```
PC = PC + (SignExtend(signed_immed_11) << 1)
```

Usage

To calculate the correct value of signed_immed_11, the assembler (or other toolkit component) needs to:

1. Form the base address for the branch. This is the address of the branch instruction, plus 4. In other words, the base address is equal to the PC value read by that instruction.
2. Subtract the base address from the target address to form a byte offset. This offset is always even, because all Thumb instructions are halfword-aligned.
3. If the byte offset is outside the range -2048 to +2046, use an alternative code-generation strategy or produce an error as appropriate.
4. Otherwise, set the signed_immed_11 field of the instruction to the byte offset divided by 2.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

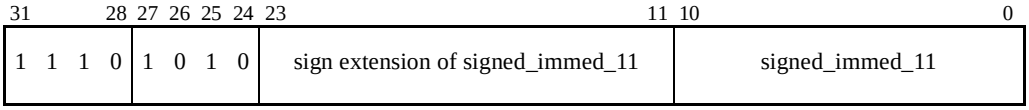
Notes

Memory bounds Branching backwards past location zero and forwards over the end of the 32-bit address space is UNPREDICTABLE.

Equivalent ARM syntax and encoding

A close equivalent is:

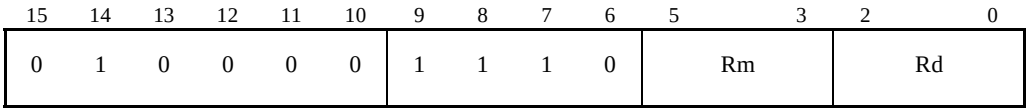
```
B  <target_address>
```



This differs from the Thumb instruction, because the offset in the ARM instruction is shifted left by 2 before being added to the PC, whereas the offset in the Thumb instruction is shifted left by 1. Also, the PC values read by the ARM and Thumb instructions are different.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.15 BIC



The `BIC` (Bit Clear) instruction performs a bitwise AND of the value of one register and the bitwise inverse of the value of another register. The condition code flags are updated, based on the result.

Syntax

```
BIC  <Rd>, <Rm>
```

where:

`<Rd>` Is the register containing the value to be ANDed, and is also the destination register for the completed operation.

`<Rm>` Specifies the register that contains the value whose complement is ANDed with the value in `<Rd>`.

Architecture version

All T variants

Exceptions

None

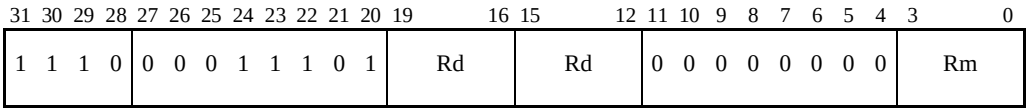
Operation

```
Rd = Rd AND NOT Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

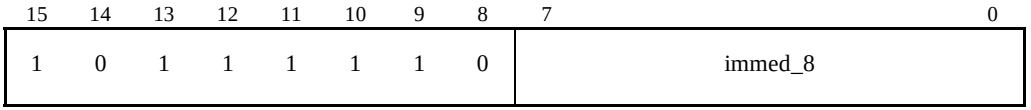
Equivalent ARM syntax and encoding

```
BICS  <Rd>, <Rd>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.16 BKPT



The BKPT (Breakpoint) instruction causes a software breakpoint to occur. This breakpoint can be handled by an exception handler installed on the prefetch abort vector. In implementations which also include debug hardware, the hardware can optionally override this behavior and handle the breakpoint itself. When this occurs, the prefetch abort vector is not entered.

Syntax

```
BKPT  <immed_8>
```

where:

`<immed_8>` Is an 8-bit immediate value, which is placed in bits[7:0] of the instruction. This value is ignored by the ARM hardware, but can be used by a debugger to store additional information about the breakpoint.

Architecture version

T variants of version 5 and above

Exceptions

Prefetch Abort

Operation

```
if (not overridden by debug hardware)
    R14_abt   = address of BKPT instruction + 4
    SPSR_abt  = CPSR
    CPSR[4:0] = 0b10111             /* Enter Abort mode */
    CPSR[5]   = 0                   /* Execute in ARM state */
    /* CPSR[6] is unchanged */
    CPSR[7]   = 1                   /* Disable normal interrupts */
    if high vectors configured then
        PC    = 0xFFFF000C
    else
        PC    = 0x0000000C
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Usage

The exact usage of the BKPT instruction depends on the debug system being used. A debug system can use the BKPT instruction in two ways:

- Debug hardware (if present) does not override the normal behavior of the BKPT instruction, and so the prefetch abort vector is entered. If the system also allows real prefetch aborts to occur, the prefetch abort handler determines (in a system-dependent manner) whether the vector entry occurred as a result of a BKPT instruction or as a result of a real prefetch abort, and branches to debug code or prefetch abort code accordingly. Otherwise, the prefetch abort handler just branches straight to debug code.

  When used in this manner, the BKPT instruction must be avoided within abort handlers, as it corrupts R14_abt and SPSR_abt. For the same reason, it must also be avoided within FIQ handlers, as an FIQ interrupt can occur within an abort handler.

- Debug hardware overrides the normal behavior of the BKPT instruction and handles the software breakpoint itself. When finished, it typically either resumes execution at the instruction following the BKPT, or replaces the BKPT in memory with another instruction and resumes execution at that instruction.

  When BKPT is used in this manner, R14_abt and SPSR_abt are not corrupted, and so the above restrictions about its use in abort and FIQ handlers do not apply.

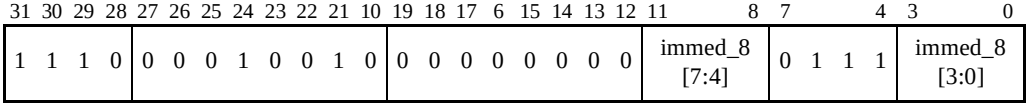
Notes

Hardware override Debug hardware in an implementation is specifically permitted to override the normal behavior of the BKPT instruction. Because of this, software must not use this instruction for purposes other than those permitted by the debug system being used (if any). In particular, software cannot rely on the Prefetch Abort exception occurring, unless either there is guaranteed to be no debug hardware in the system or the debug system specifies that it will occur.

For more information, consult the documentation for the debug system being used.

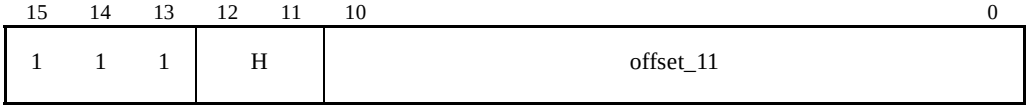
Equivalent ARM syntax and encoding

```
BKPT  <immed_8>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.17 BL, BLX(1)



This form of the BL (Branch with Link) instruction provides an unconditional subroutine call to another Thumb routine. The return from subroutine is typically performed by one of the following:

- `MOV PC,LR`
- `BX LR`
- a `POP` instruction that loads the PC.

This form of the BLX (Branch with Link and Exchange instruction set) instruction provides an unconditional subroutine call to an ARM routine. The return from subroutine is typically performed by a `BX LR` instruction, or an `LDR` or `LDM` instruction that loads the PC.

To allow for a reasonably large offset to the target subroutine, each of these two instructions is automatically translated by the assembler into a sequence of two 16-bit Thumb instructions:

- The first Thumb instruction has H == 10 and supplies the high part of the branch offset. This instruction sets up for the subroutine call and is shared between the BL and BLX forms.
- The second Thumb instruction has H == 11 (for BL) or H == 01 (for BLX). It supplies the low part of the branch offset and causes the subroutine call to take place.

Syntax

```
BL   <target_addr>
BLX  <target_addr>
```

where:

`<target_addr>` Specifies the address to branch to. The branch target address is calculated by:

1. Shifting the offset_11 field of the first instruction left twelve bits.
2. Sign-extending the result to 32 bits.
3. Adding this to the contents of the PC (which contains the address of the first instruction plus 4).
4. Adding twice the offset_11 field of the second instruction. For BLX, the resulting address is forced to be word-aligned by clearing bit[1].

The instruction can therefore specify a branch of approximately ±4MB.

Architecture version

BL (H == 10 and H == 11 forms) is in all T variants.

BLX (H == 01 form) is in T variants of version 5 and above.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Exceptions

None

Operation

```
if H == 10 then
    LR = PC + (SignExtend(offset_11) << 12)

else if H == 11 then
    PC = LR + (offset_11 << 1)
    LR = (address of next instruction) | 1

else if H == 01 then
    PC = (LR + (offset_11 << 1)) AND 0xFFFFFFFC
    LR = (address of next instruction) | 1
    T Flag = 0
```

Usage

To generate the correct pair of instructions, the assembler (or other toolkit component) must first generate the branch offset, as follows:

1. Form the base address for the branch. This is the address of the first of the two Thumb instructions (the one with H == 10), plus 4. In other words, the base address is equal to the PC value read by that instruction.
2. If the instruction is BLX, set bit[1] of the target address to be equal to bit[1] of the base address. This is an exception to the normal rule that bits[1:0] of the address of an ARM instruction are 0b00. This adjustment is required to ensure that the restrictions associated with the H == 01 form of the instruction are obeyed.
3. Subtract the base address from the target address to form the offset.

The resulting offset is always even. If the offset lies outside the range:

$$-2^{22} <= \text{offset} <= +2^{22} - 2$$

the target address lies outside the addressing range of these instructions. This results in alternative code or an error, as appropriate.

If the offset is in range, a sequence of two Thumb instructions must be generated, both using the above form:

- The first with H == 10 and offset_11 = offset[22:12].
- The second with H == 11 (for BL) or H== 01 (for BLX) and offset_11 = offset[11:1].

Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

Encoding If H == 00, the instruction is an unconditional branch instruction instead (see the Thumb instruction *B (2)* on page A7-20).

Bit[0] for BLX If H == 01, then bit[0] of the instruction must be zero, or the instruction is UNDEFINED. The offset calculation method described in *Usage* above ensures that the offset calculated for a BLX instruction is a multiple of four, and that this restriction is obeyed.

Memory bounds Branching backwards past location zero and forwards over the end of the 32-bit address space is UNPREDICTABLE.

Instruction pairs These Thumb instructions must always occur in the pairs described above. Specifically:

- If a Thumb instruction at address A is the H==10 form of this instruction, the Thumb instruction at address A+2 must be either the H==01 or the H==11 form of this instruction.
- If a Thumb instruction at address A is either the H==01 or the H==11 form of this instruction, the Thumb instruction at address A-2 must be the H==10 form of this instruction.

Also, except as noted below under *Exceptions*, the second instruction of the pair must not be the target of any branch, whether as the result of a branch instruction or of some other instruction that changes the PC.

Failure to adhere to any of these restrictions can result in UNPREDICTABLE behavior.

Exceptions It is IMPLEMENTATION DEFINED whether processor exceptions can occur between the two instructions of a BL or BLX pair. If they can, the ARM instructions designed for use for exception returns must be capable of returning correctly to the second instruction of the pair. So, exception handlers need take no special precautions about returning to the second instruction of a BL or BLX pair.

Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

Close equivalents to these instruction pairs are as follows.

To call a Thumb subroutine:

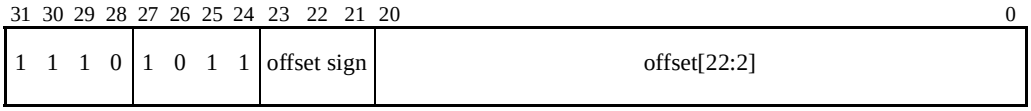
```
BLX  <target_addr>
```



where L == offset[1].

To call an ARM routine:

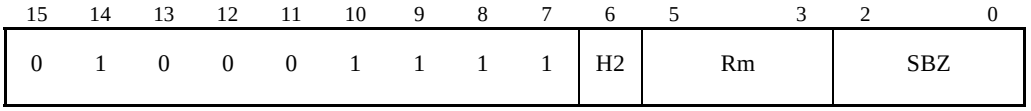
```
BL  <target_addr>
```



These differ slightly from the Thumb instruction pairs because of the different values of the PC in ARM and Thumb code. This can be compensated for by adjusting the offset by 4.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.18 BLX(2)



15	14	13	12	11	10	9	8	7	6	5 3	2 0
0	1	0	0	0	1	1	1	1	H2	Rm	SBZ

This form of BLX is used to call an ARM or Thumb subroutine from the Thumb instruction set, at an address specified in a register. This instruction branches and selects the instruction decoder to use to decode the instructions at the branch destination.

The T flag is updated with bit[0] of the value of register Rm. To return from the subroutine to the caller, use `BX R14`.

Syntax

```
BLX  <Rm>
```

where:

`<Rm>` Is the register that contains the branch target address. It can be any of R0 to R14. The register number is encoded in the instruction in H2 (most significant bit) and Rm (remaining three bits). If R15 is specified for `<Rm>`, the results are UNPREDICTABLE.

Architecture version

T variants of version 5 and above

Exceptions

None

Operation

```
LR = (address of the instruction after this BLX) | 1
T Flag = Rm[0]
PC = Rm[31:1] << 1
```

Notes

Encoding Bit 7 is the H1 bit for some of the other instructions that access the high registers. If it is 0 for this instruction, rather than 1 as shown, the instruction is a BX instruction instead (see *BX* on page A7-32).

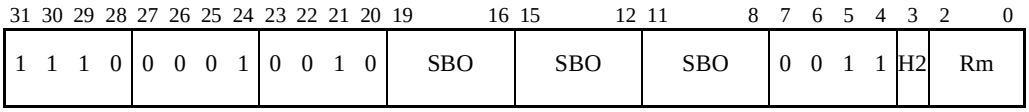
ARM/Thumb state transfers

If Rm[1:0] == 0b10, the result is UNPREDICTABLE, as branches to non word-aligned addresses are impossible in ARM state.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

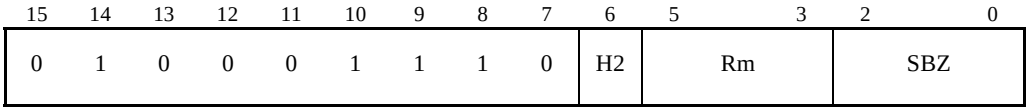
Equivalent ARM syntax and encoding

```
BLX  <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.19 BX



The BX (Branch and Exchange) instruction is used to branch between ARM code and Thumb code.

Syntax

```
BX  <Rm>
```

where:

<Rm> Is the register that contains the branch target address. It can be any of R0 to R15. The register number is encoded in the instruction in H2 (most significant bit) and Rm (remaining three bits).

Architecture version

All T variants

Exceptions

None

Operation

```
T Flag = Rm[0]
PC = Rm[31:1] << 1
```

Usage

The normal subroutine return instruction in Thumb code is `BX R14`. The following subroutine call instructions leave a suitable return value in R14:

- ARM BLX instructions (See *BLX (1)* on page A4-16 and *BLX (2)* on page A4-18)
- Thumb BL and BLX instructions (see *BL, BLX(1)* on page A7-26 and *BLX(2)* on page A7-30).

In T variants of ARM architecture version 4, a subroutine call to an ARM routine can be performed by a code sequence of the form:

```
<Put address of routine to call in Ra>
MOV   LR,PC        ; Return to second following instruction
BX    Ra
```

In T variants of ARM architecture 5 and above, a subroutine call to an ARM routine can be performed more efficiently with a BLX instruction (see *BL, BLX(1)* on page A7-26 and *BLX(2)* on page A7-30).

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

Encoding Bit 7 is the H1 bit for some of the other instructions that access the high registers. If it is 1 for this instruction, rather than 0 as shown, the instruction is:

- a BLX instruction instead in ARM architecture version 5 and above (see *BLX(2)* on page A7-30)
- UNPREDICTABLE prior to ARM architecture version 5.

ARM/Thumb state transfers

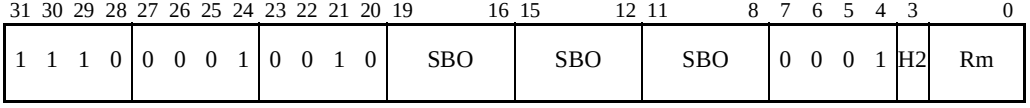
If Rm[1:0] == 0b10, the result is UNPREDICTABLE, as branches to non word-aligned addresses are impossible in ARM state.

Use of R15 Register 15 can be specified for <Rm>. If this is done, R15 is read as normal for Thumb code, that is, it is the address of the BX instruction itself plus 4. If the BX instruction is at a word-aligned address, this results in a branch to the next word, executing in ARM state. However, if the BX instruction is not at a word-aligned address, this means that the results of the instruction are UNPREDICTABLE (because the value read for R15 has bits[1:0]==0b10).

Equivalent ARM syntax and encoding

A close equivalent is:

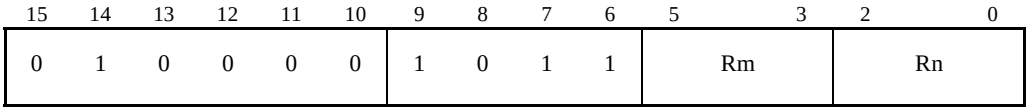
```
BX   <Rm>
```



This ARM instruction is not quite equivalent to the Thumb instruction, because their specified behavior differs when <Rm> is R15.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.20 CMN



The `CMN` (Compare Negative) instruction compares a register value with the negation of another register value. The condition flags are updated, based on the result of adding the two register values, so that subsequent instructions can be conditionally executed (using a conditional branch).

Syntax

```
CMN  <Rn>, <Rm>
```

where:

`<Rn>` Is the register containing the first value for comparison.

`<Rm>` Is the register containing the second value for comparison.

Architecture version

All T variants

Exceptions

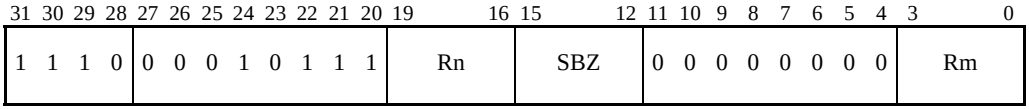
None

Operation

```
alu_out = Rn + Rm
N Flag = alu_out[31]
Z Flag = if alu_out == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn + Rm)
V Flag = OverflowFrom(Rn + Rm)
```

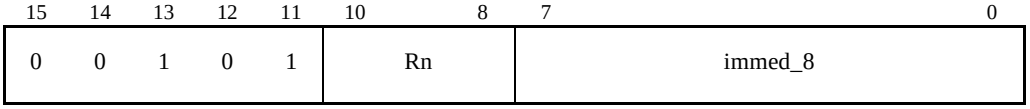
Equivalent ARM syntax and encoding

```
CMN  <Rn>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.21 CMP (1)



This form of the CMP (Compare) instruction compares a register value with a large immediate value. The condition flags are updated, based on the result of subtracting the constant from the register value, so that subsequent instructions can be conditionally executed (using a conditional branch).

Syntax

```
CMP  <Rn>, #<immed_8>
```

where:

`<Rn>` Is the register containing the first value for comparison.

`<immed_8>` Is the 8-bit second value for comparison.

Architecture version

All T variants

Exceptions

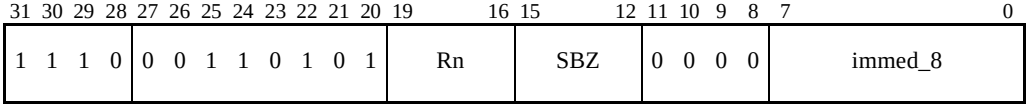
None

Operation

```
alu_out = Rn - immed_8
N Flag = alu_out[31]
Z Flag = if alu_out == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn - immed_8)
V Flag = OverflowFrom(Rn - immed_8)
```

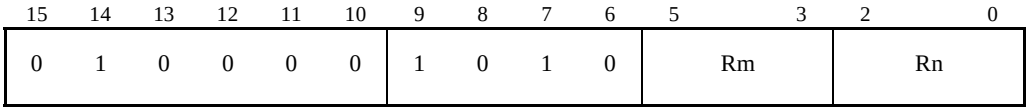
Equivalent ARM syntax and encoding

```
CMP  <Rn>, #<immed_8>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.22 CMP (2)



This form of CMP compares two register values. The condition code flags are updated, based on the result of subtracting the second register value from the first, so that subsequent instructions can be conditionally executed (using a conditional branch).

Syntax

```
CMP  <Rn>, <Rm>
```

where:

<Rn> Is the register containing the first value for comparison.

<Rm> Is the register containing the second value for comparison.

Architecture version

All T variants

Exceptions

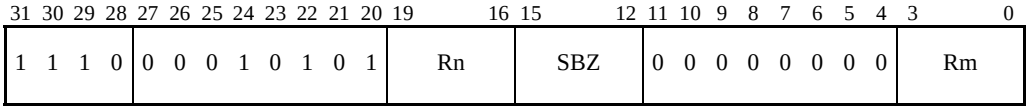
None

Operation

```
alu_out = Rn - Rm
N Flag = alu_out[31]
Z Flag = if alu_out == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn - Rm)
V Flag = OverflowFrom(Rn - Rm)
```

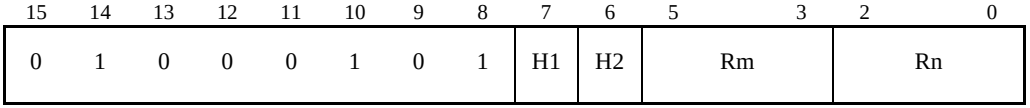
Equivalent ARM syntax and encoding

```
CMP  <Rn>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.23 CMP (3)



15	14	13	12	11	10	9	8	7	6	5 3	2 0
0	1	0	0	0	1	0	1	H1	H2	Rm	Rn

This form of CMP compares the values of two registers, one or both of which are high registers. The condition flags are updated, based on the result of subtracting the second register value from the first, so that subsequent instructions can be conditionally executed (using a conditional branch).

Syntax

```
CMP  <Rn>, <Rm>
```

where:

<Rn> Is the register containing the first value. It can be any of R0 to R15. Its number is encoded in the instruction in H1 (most significant bit) and Rn (remaining three bits).

<Rm> Is the register containing the second value. It can be any of R0 to R15. Its number is encoded in the instruction in H2 (most significant bit) and Rm (remaining three bits).

Architecture version

All T variants

Exceptions

None

Operation

```
alu_out = Rn - Rm
N Flag = alu_out[31]
Z Flag = if alu_out == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn - Rm)
V Flag = OverflowFrom(Rn - Rm)
```

Notes

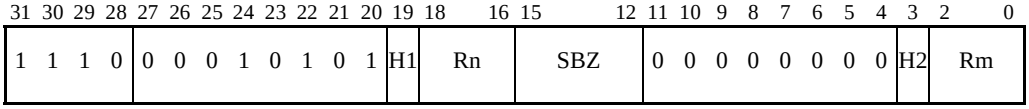
Operand restriction If a low register is specified for both <Rn> and <Rm> (H1==0 and H2==0), the result is UNPREDICTABLE.

Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

A close equivalent is:

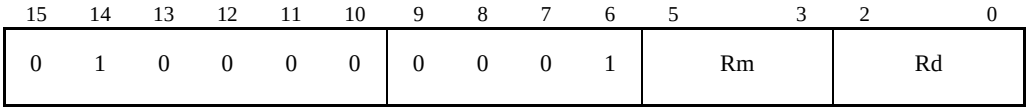
```
CMP  <Rn>, <Rm>
```



There are slight differences when the instruction accesses the PC, because of the different definitions of the PC when executing ARM and Thumb code.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.24 EOR



The EOR (Exclusive OR) instruction performs a bitwise EOR of the values in two registers. The condition code flags are updated, based on the result.

Syntax

```
EOR  <Rd>, <Rm>
```

where:

<Rd> Specifies the register containing the first operand, and is also the destination register.

<Rm> Specifies the register containing the second operand.

Architecture version

All T variants

Exceptions

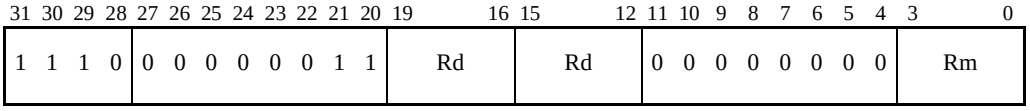
None

Operation

```
Rd = Rd EOR Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

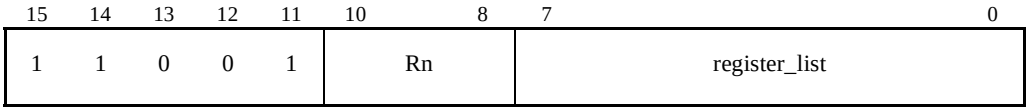
Equivalent ARM syntax and encoding

```
EORS  <Rd>, <Rd>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.25 LDMIA



The LDMIA (Load Multiple Increment After) instruction loads a non-empty subset, or possibly all, of the general-purpose registers from sequential memory locations.

Syntax

```
LDMIA  <Rn>!, <registers>
```

where:

`<Rn>` Is the register containing the start address for the instruction.

`!` Causes base register writeback, and is not optional.

`<registers>` Is a list of registers to be loaded, separated by commas and surrounded by { and }. The list is encoded in the register_list field of the instruction, by setting bit[i] to 1 if register Ri is included in the list and to 0 otherwise, for each of i=0 to 7.

At least one register must be loaded. If bits[7:0] are all zero, the result is UNPREDICTABLE.

The registers are loaded in sequence, the lowest-numbered register from the lowest memory address (`start_address`), through to the highest-numbered register from the highest memory address (`end_address`).

The `start_address` is the value of the base register `<Rn>`. Subsequent addresses are formed by incrementing the previous address by four. One address is produced for each register that is specified in `<registers>`.

The `end_address` value is four less than the sum of the value of the base register and four times the number of registers specified in `<registers>`.

Finally, the base register `<Rn>` is incremented by four times the numbers of registers in `<registers>`.

Architecture version

All T variants

Exceptions

Data Abort

Copyright © 1996-2000 ARM Limited. All rights reserved.

Operation

```
start_address = Rn
end_address = Rn + (Number_Of_Set_Bits_In(register_list) * 4) - 4
address = start_address
for i = 0 to 7
    if register_list[i] == 1
        Ri = Memory[address,4]
        address = address + 4
assert end_address == address - 4
Rn = Rn + (Number_Of_Set_Bits_In(register_list) * 4)
```

Usage

The `LDMIA` instruction is useful as a block load instruction. Combined with `STMIA` (Store Multiple), it allows efficient block copy.

Notes

Operand restrictions

If the base register `<Rn>` is specified in `<registers>`, the final value of `<Rn>` is the loaded value (not the written-back value).

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment Load Multiple instructions ignore the least significant two bits of `address`.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception.

Time order The time order of the accesses to individual words of memory generated by this instruction is only defined in some circumstances. See *Data accesses to memory-mapped I/O* on page A2-32 for details.

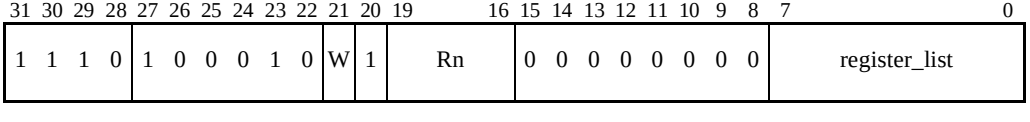
Equivalent ARM syntax and encoding

If `<Rn>` is not in the register list (W == 1):

```
LDMIA  <Rn>!, <registers>
```

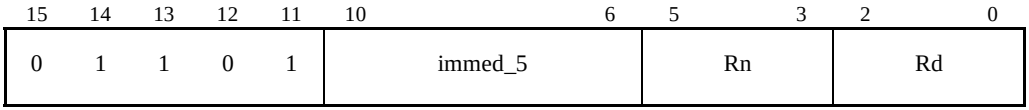
If `<Rn>` is in the register list (W == 0):

```
LDMIA  <Rn>, <registers>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.26 LDR (1)



This form of the LDR (Load Register) instruction allows 32-bit memory data to be loaded into a general-purpose register. The addressing mode is useful for accessing structure (record) fields. With an offset of zero, the address produced is the unaltered value of the base register <Rn>.

Syntax

```
LDR  <Rd>, [<Rn>, #<immed_5> * 4]
```

where:

<Rd> Is the destination register for the word loaded from memory.

<Rn> Is the register containing the base address for the instruction.

<immed_5> Is a 5-bit value that is multiplied by 4 and added to the value of <Rn> to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + (immed_5 * 4)
if address[1:0] == 0b00
    data = Memory[address,4]
else
    data = UNPREDICTABLE
Rd = data
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

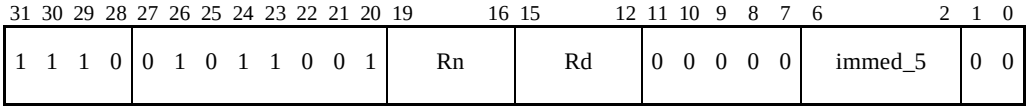
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not word-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

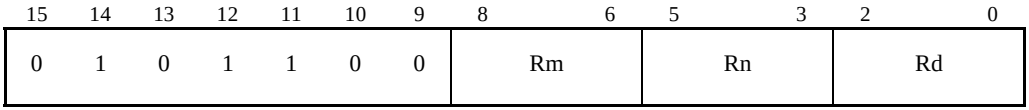
Equivalent ARM syntax and encoding

```
LDR  <Rd>, [<Rn>, #<immed_5> * 4]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.27 LDR (2)



This form of LDR allows 32-bit memory data to be loaded into a general-purpose register. The addressing mode is useful for pointer+large offset arithmetic and for accessing a single element of an array.

Syntax

```
LDR  <Rd>, [<Rn>, <Rm>]
```

where:

<Rd> Is the destination register for the word loaded from memory.

<Rn> Is the register containing the first value used in forming the memory address.

<Rm> Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + Rm
if address[1:0] == 0b00
    data = Memory[address,4]
else
    data = UNPREDICTABLE
Rd = data
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

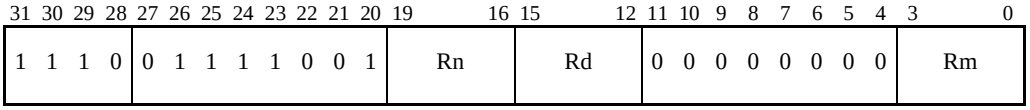
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not word-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

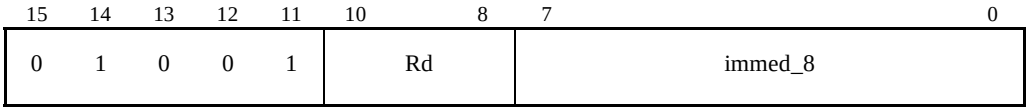
Equivalent ARM syntax and encoding

```
LDR  <Rd>, [<Rn>, <Rm>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.28 LDR (3)



This form of LDR allows 32-bit memory data to be loaded into a general-purpose register. The addressing mode is useful for accessing PC-relative data.

Syntax

```
LDR  <Rd>, [PC, #<immed_8> * 4]
```

where:

`<Rd>` Is the destination register for the word loaded from memory.

`PC` Is the program counter. Its value is used to calculate the memory address. Bit 1 of the PC value is forced to zero for the purpose of this calculation, so the address is always word-aligned.

`<immed_8>` Is an 8-bit value that is multiplied by 4 and added to the value of the PC to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = (PC[31:2] << 2) + (immed_8 * 4)
Rd = Memory[address, 4]
```

Notes

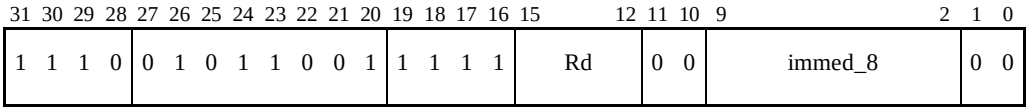
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

A close equivalent is:

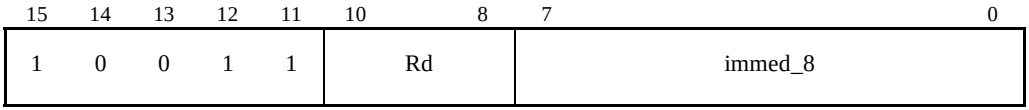
```
LDR  <Rd>, [PC, #<immed_8> * 4]
```



There are slight differences caused by the different definitions of the PC and the fact that the Thumb instruction ignores bit[1] of the PC.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.29 LDR (4)



This form of LDR allows 32-bit memory data to be loaded into a general-purpose register. The addressing mode is useful for accessing stack data.

Syntax

```
LDR  <Rd>, [SP, #<immed_8> * 4]
```

where:

`<Rd>` Is the destination register for the word loaded from memory.

`SP` Is the stack pointer. Its value is used to calculate the memory address.

`<immed_8>` Is an 8-bit value that is multiplied by 4 and added to the value of the SP to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = SP + (immed_8 * 4)
if address[1:0] == 0b00
    data = Memory[address,4]
else
    data = UNPREDICTABLE
Rd = data
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

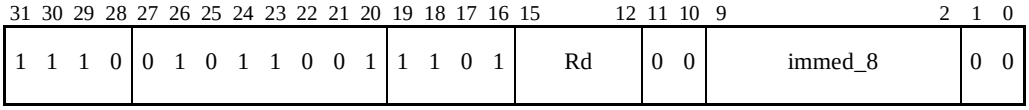
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not word-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

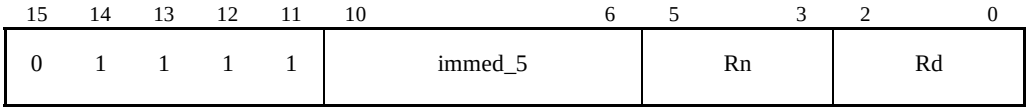
Equivalent ARM syntax and encoding

```
LDR  <Rd>, [SP, #<immed_8> * 4]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.30 LDRB (1)



This form of the `LDRB` (Load Register Byte) instruction loads a byte from memory and zero-extends it to form a 32-bit word which is written to a general-purpose register. The addressing mode is useful for accessing structure (record) fields. With an offset of zero, the address produced is the unaltered value of the base register `<Rn>`.

Syntax

```
LDRB  <Rd>, [<Rn>, #<immed_5>]
```

where:

`<Rd>` Is the destination register for the byte loaded from memory.

`<Rn>` Is the register containing the base address for the instruction.

`<immed_5>` Is a 5-bit value that is added to the value of `<Rn>` to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

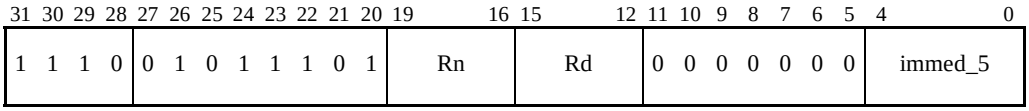
```
address = Rn + immed_5
Rd = Memory[address,1]
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

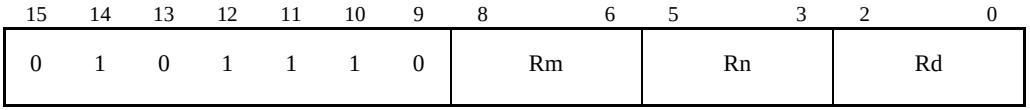
Equivalent ARM syntax and encoding

```
LDRB  <Rd>, [<Rn>, #<immed_5>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.31 LDRB (2)



This form of LDRB loads a byte from memory and zero-extends it to form a 32-bit word, which is written to a general-purpose register. The addressing mode is useful for pointer+large offset arithmetic and for accessing a single element of an array.

Syntax

```
LDRB  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the destination register for the byte loaded from memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

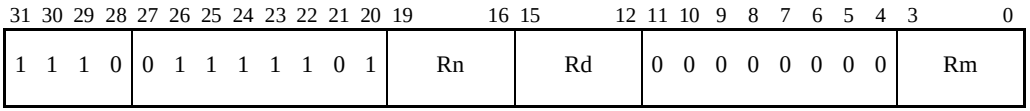
```
address = Rn + Rm
Rd = Memory[address,1]
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

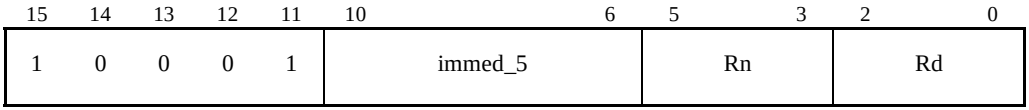
Equivalent ARM syntax and encoding

```
LDRB  <Rd>, [<Rn>, <Rm>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.32 LDRH (1)



This form of the LDRH (Load Register Halfword) instruction loads a halfword (16 bits) from memory and zero-extends it to form a 32-bit word, which is written to a general-purpose register. The addressing mode is useful for accessing structure (record) fields. With an offset of zero, the address produced is the unaltered value of the base register <Rn>.

Syntax

```
LDRH  <Rd>, [<Rn>, #<immed_5> * 2]
```

where:

<Rd> Is the destination register for the halfword loaded from memory.

<Rn> Is the register containing the base address for the instruction.

<immed_5> Is a 5-bit value that is multiplied by 2, then added to the value of <Rn> to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + (immed_5 * 2)
if address[0] == 0
    data = Memory[address,2]
else
    data = UNPREDICTABLE
Rd = data
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

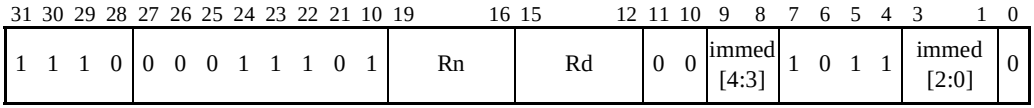
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not halfword-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bit[0] != 0 causes an alignment exception (a type of data abort).

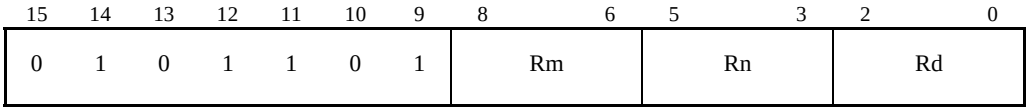
Equivalent ARM syntax and encoding

```
LDRH  <Rd>, [<Rn>, #<immed_5> * 2]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.33 LDRH (2)



This form of LDRH loads a halfword (16 bits) from memory and zero-extends it to form a 32-bit word, which is written to a general-purpose register. The addressing mode is useful for pointer + large offset arithmetic and for accessing a single element of an array.

Syntax

```
LDRH  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the destination register for the halfword loaded from memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + Rm
if address[0] == 0
    data = Memory[address,2]
else
    data = UNPREDICTABLE
Rd = data
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

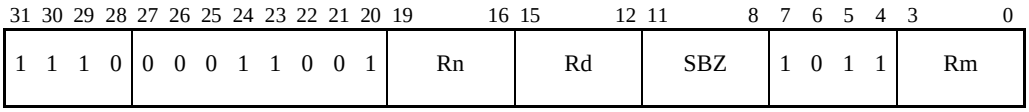
Alignment If the memory address is not halfword-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bit[0] != 0 causes an alignment exception (a type of data abort).

 Copyright © 1996-2000 ARM Limited. All rights reserved.

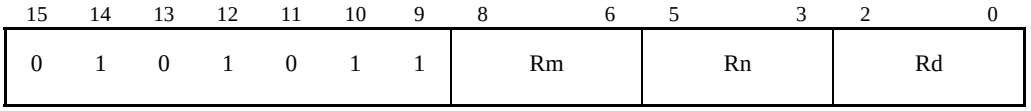
Equivalent ARM syntax and encoding

```
LDRH  <Rd>, [<Rn>, <Rm>]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.34 LDRSB



The LDRSB (Load Register Signed Byte) instruction loads a byte from memory and sign-extends it to form a 32-bit word, which is written to a general-purpose register.

Syntax

```
LDRSB  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the destination register for the byte loaded from memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

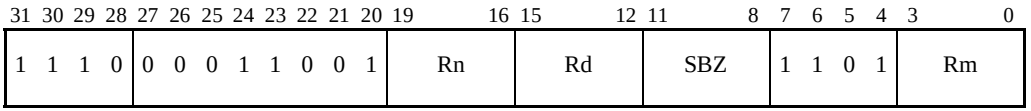
```
address = Rn + Rm
Rd = SignExtend(Memory[address,1])
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

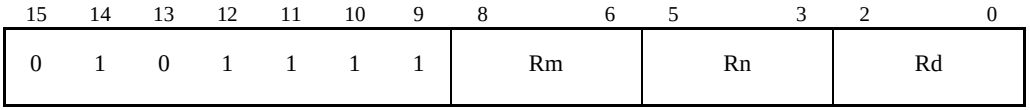
Equivalent ARM syntax and encoding

```
LDRSB  <Rd>, [<Rn>, <Rm>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.35 LDRSH



The LDRSH (Load Register Signed Halfword) instruction loads a halfword from memory and sign-extends it to form a 32-bit word, which is written to a general-purpose register.

Syntax

```
LDRSH  <Rd>, [<Rn>, <Rm>]
```

where:

<Rd> Is the destination register for the halfword loaded from memory.

<Rn> Is the register containing the first value used in forming the memory address.

<Rm> Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + Rm
if address[0] == 0
    data = Memory[address,2]
else
    data = UNPREDICTABLE
Rd = SignExtend(data)
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

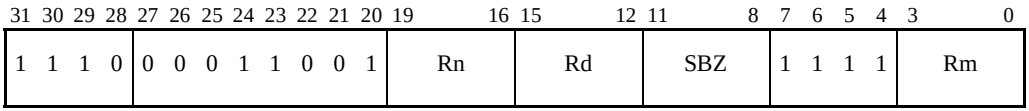
Alignment If the memory address is not halfword-aligned and no data abort occurs, the value written to the destination register is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bit[0] != 0 causes an alignment exception (a type of data abort).

 Copyright © 1996-2000 ARM Limited. All rights reserved.

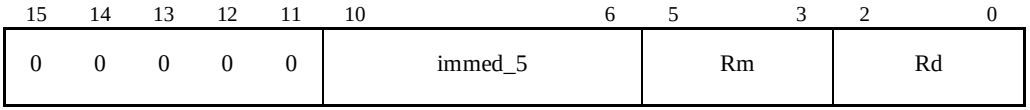
Equivalent ARM syntax and encoding

```
LDRSH  <Rd>, [<Rn>, <Rm>]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.36 LSL (1)



This form of the LSL (Logical Shift Left) instruction is used to provide either the value of a register directly (`LSL #0`), or the value of a register multiplied by a constant power of two. Zeros are inserted into the bit positions vacated by the shift, and the condition code flags are updated, based on the result.

Syntax

```
LSL  <Rd>, <Rm>, #<immed_5>
```

where:

`<Rd>` Is the register that stores the result of the operation.

`<Rm>` Is the register containing the value to be shifted.

`<immed_5>` Specifies the shift amount, in the range 0 to 31.

Architecture version

All T variants

Exceptions

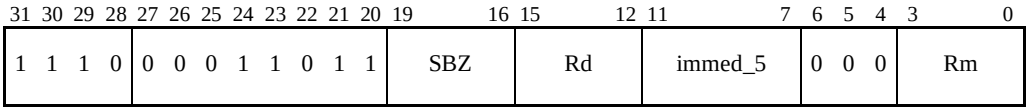
None

Operation

```
if immed_5 == 0
    C Flag = unaffected
    Rd = Rm
else /* immed_5 > 0 */
    C Flag = Rm[32 - immed_5]
    Rd = Rm Logical_Shift_Left immed_5
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```

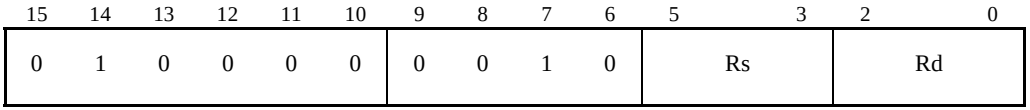
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rm>, LSL #<immed_5>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.37 LSL (2)



This form of LSL is used to provide the value of a register multiplied by a variable power of two. Zeros are inserted into the vacated bit positions. The condition code flags are updated, based on the result.

Syntax

```
LSL  <Rd>, <Rs>
```

where:

<Rd> Contains the value to be shifted, and is the destination register for the result of the operation.

<Rs> Is the register containing the shift value. The value is held in the least significant byte.

Architecture version

All T variants

Exceptions

None

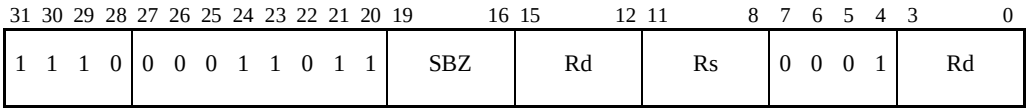
Operation

```
if Rs[7:0] == 0
    C Flag = unaffected
    Rd = unaffected
else if Rs[7:0] < 32 then
    C Flag = Rd[32 - Rs[7:0]]
    Rd = Rd Logical_Shift_Left Rs[7:0]
else if Rs[7:0] == 32 then
    C Flag = Rd[0]
    Rd = 0
else /* Rs[7:0] > 32 */
    C Flag = 0
    Rd = 0
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

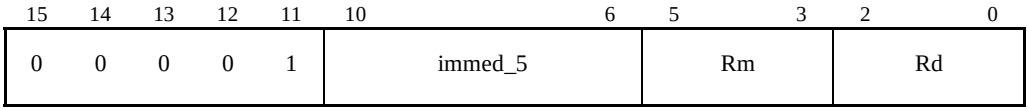
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rd>, LSL <Rs>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.38 LSR (1)



This form of the LSR (Logical Shift Right) instruction is used to provide the unsigned value of a register, divided by a constant power of two. LSR performs a logical shift right of the value of register <Rm>, and zeros are inserted into the vacated bit positions. The condition code flags are updated, based on the result.

Syntax

```
LSR  <Rd>, <Rm>, #<immed_5>
```

where:

<Rd> Is the destination register for the operation.

<Rm> Is the register containing the value to be shifted.

<immed_5> Specifies the shift amount, in the range 1 to 32. Shifts by 1 to 31 are encoded directly in immed_5. A shift by 32 is encoded as immed_5 == 0.

Architecture version

All T variants

Exceptions

None

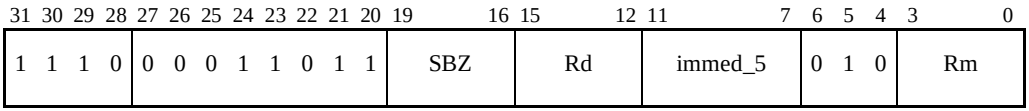
Operation

```
if immed_5 == 0
    C Flag = Rd[31]
    Rd = 0
else /* immed_5 > 0 */
    C Flag = Rd[immed_5 - 1]
    Rd = Rm Logical_Shift_Right immed_5
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

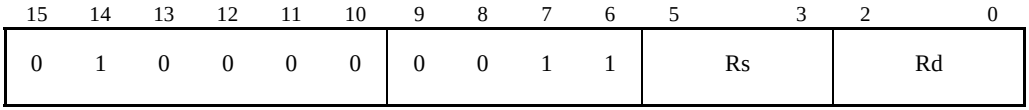
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rm>, LSR #<immed_5>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.39 LSR (2)



This form of LSR is used to provide the unsigned value of a register divided by a variable power of two. Zeros are inserted into the vacated bit positions. The condition code flags are updated, based on the result.

Syntax

```
LSR  <Rd>, <Rs>
```

where:

`<Rd>` Contains the value to be shifted, and is the destination register for the result of the operation.

`<Rs>` Is the register containing the shift value. The value is held in the least significant byte.

Architecture version

All T variants

Exceptions

None

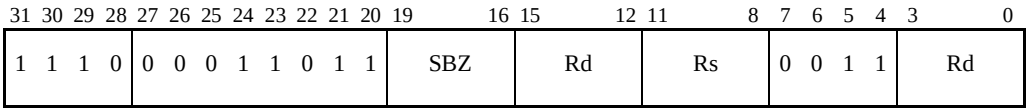
Operation

```
if Rs[7:0] == 0 then
    C Flag = unaffected
    Rd = unaffected
else if Rs[7:0] < 32 then
    C Flag = Rd[Rs[7:0] - 1]
    Rd = Rd Logical_Shift_Right Rs[7:0]
else if Rs[7:0] == 32 then
    C Flag = Rd[31]
    Rd = 0
else /* Rs[7:0] > 32 */
    C Flag = 0
    Rd = 0
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

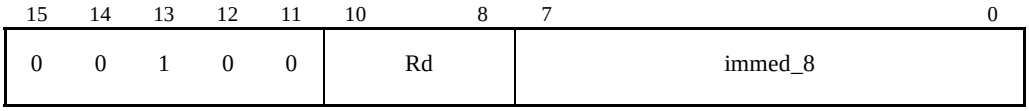
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rd>, LSR <Rs>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.40 MOV (1)



This form of the `MOV` (Move) instruction moves a large immediate value to a register. The condition code flags are updated, based on the result.

Syntax

```
MOV  <Rd>, #<immed_8>
```

where:

`<Rd>` Is the destination register for the operation.

`<immed_8>` Is an 8-bit immediate value, in the range 0 to 255, to move into `<Rd>`.

Architecture version

All T variants

Exceptions

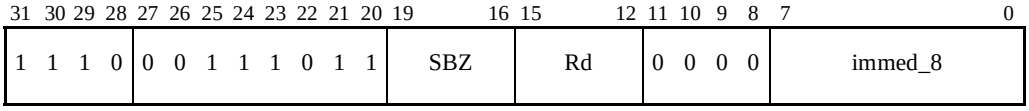
None

Operation

```
Rd = immed_8
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

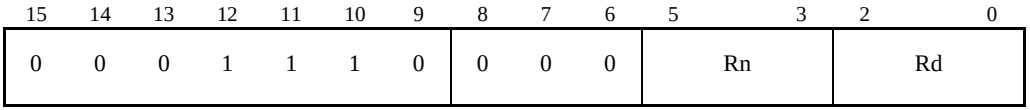
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, #<immed_8>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.41 MOV (2)



This form of `MOV` is used to move a value from one low register to another, and the flags are set according to that value.

Syntax

```
MOV  <Rd>, <Rn>
```

where:

`<Rd>` Is the destination register for the operation.

`<Rn>` Is the register containing the value to be copied.

Architecture Version

All T variants

Exceptions

None

Operation

```
Rd = Rn
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = 0
V Flag = 0
```

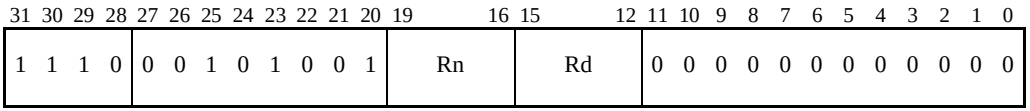
Notes

Encoding This instruction is encoded as `ADD Rd, Rn, #0`.

See also *ADD (1)* on page A7-5.

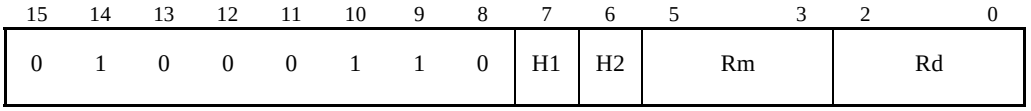
Equivalent ARM syntax and encoding

```
ADDS  <Rd>, <Rn>, #0
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.42 MOV (3)



This form of `MOV` is used to move a value to, from, or between high registers. Unlike the low register `MOV` instruction described in *MOV (2)* on page A7-67, this instruction does not change the flags.

Syntax

```
MOV  <Rd>, <Rm>
```

where:

`<Rd>` Is the destination register for the operation. It can be any of R0 to R15, and its number is encoded in the instruction in H1 (most significant bit) and Rd (remaining three bits).

`<Rm>` Is the register containing the value to be copied. It can be any of R0 to R15, and its number is encoded in the instruction in H2 (most significant bit) and Rm (remaining three bits).

Architecture version

All T variants

Exceptions

None

Operation

```
Rd = Rm
```

Usage

The instruction `MOV PC,R14` can be used as a subroutine return instruction if it is known that the caller is also a Thumb routine. However, it is more usual to use `BX R14` (see *BX* on page A7-32), which works regardless of whether the caller is an ARM routine or a Thumb routine.

In T variants of ARM architecture 4, the instruction `MOV R14,PC` is often used to set up a return link value in R14 for a call to an ARM routine. In T variants of ARM architecture 5 and above, there are more efficient ways of doing this. See *BX* on page A7-32 for more details.

Notes

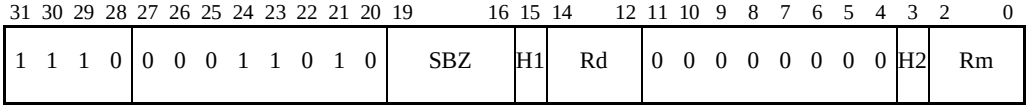
Operand restriction If a low register is specified for `<Rd>` and `<Rm>` (H1==0 and H2==0), the result is UNPREDICTABLE.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Equivalent ARM syntax and encoding

A close equivalent is:

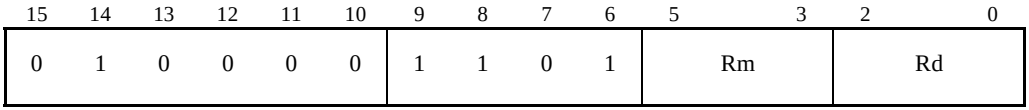
```
MOV  <Rd>, <Rm>
```



There are slight differences when the instruction accesses the PC, because of the different definitions of the PC when executing ARM and THUMB code.

Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.43 MUL



The `MUL` (Multiply) instruction multiplies signed or unsigned variables to produce a 32-bit result. The condition code flags are updated, based on the result.

Syntax

```
MUL  <Rd>, <Rm>
```

where:

`<Rd>` Contains the value to be multiplied with the value of `<Rm>`, and is also the destination register for the operation.

`<Rm>` Is the register containing the value to be multiplied with the value of `<Rd>`.

Architecture version

All T variants

Exceptions

None

Operation

```
Rd = (Rm * Rd)[31:0]
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected    /* See "C flag" note */
V Flag = unaffected
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

Operand restriction Specifying the same register for `<Rd>` and `<Rm>` has UNPREDICTABLE results.

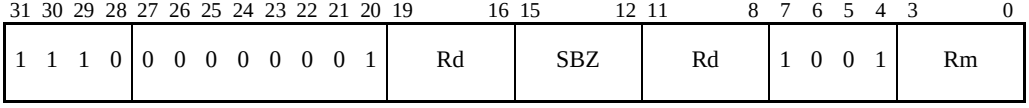
Early termination If the multiplier implementation supports early termination, it must be implemented on the value of the `<Rd>` operand. The type of early termination used (signed or unsigned) is IMPLEMENTATION DEFINED.

Signed and unsigned As the `MUL` instruction produces only the lower 32 bits of the 64-bit product, `MUL` gives the same answer for multiplication of both signed and unsigned numbers.

C flag The `MUL` instruction is defined to leave the C flag unchanged in ARM architecture version 5 and above. In earlier versions of the architecture, the value of the C flag was UNPREDICTABLE after a `MUL` instruction.

Equivalent ARM syntax and encoding

```
MULS  <Rd>, <Rm>, <Rd>
```



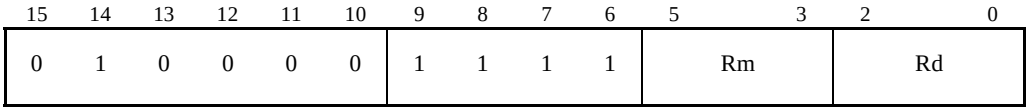
Note

The following instruction is not a suitable alternative, as it violates the operand restriction on the ARM instruction (see *MUL* on page A4-66) and might have the wrong early termination behavior:

```
MULS  <Rd>, <Rd>, <Rm>
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.44 MVN



The MVN (Move NOT) instruction is used to complement a register value, often to form a bit mask. The condition code flags are updated, based on the result.

Syntax

```
MVN  <Rd>, <Rm>
```

where:

<Rd> Is the destination register for the operation.

<Rm> Is the register containing the value whose one's complement is written to <Rd>.

Architecture version

All T variants

Exceptions

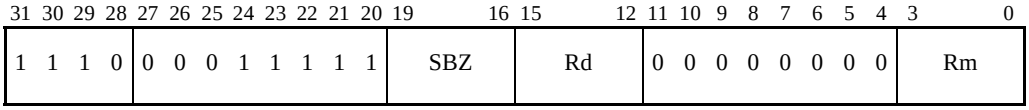
None

Operation

```
Rd = NOT Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

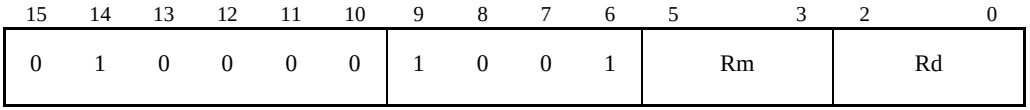
Equivalent ARM syntax and encoding

```
MVNS  <Rd>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.45 NEG



The NEG (Negate) instruction negates the value of one register and stores the result in a second register. The condition code flags are updated (based on the result).

Syntax

```
NEG  <Rd>, <Rm>
```

where:

<Rd> Is the destination register for the operation.

<Rm> Is the register containing the value that is subtracted from zero.

Architecture version

All T variants

Exceptions

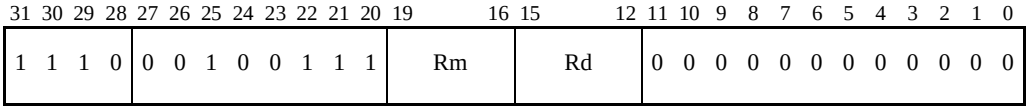
None

Operation

```
Rd = 0 - Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = NOT BorrowFrom(0 - Rm)
V Flag = OverflowFrom(0 - Rm)
```

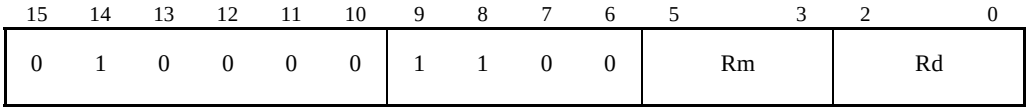
Equivalent ARM syntax and encoding

```
RSBS  <Rd>, <Rm>, #0
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.46 ORR



The `ORR` (Logical OR) instruction performs a bitwise OR of the values in two registers. The condition code flags are updated, based on the result.

Syntax

```
ORR  <Rd>, <Rm>
```

where:

`<Rd>` Is the destination register for the operation.

`<Rm>` Is the register containing the value that is ORed with the value of `<Rd>`. The operation is a bitwise inclusive OR.

Architecture version

All T variants

Exceptions

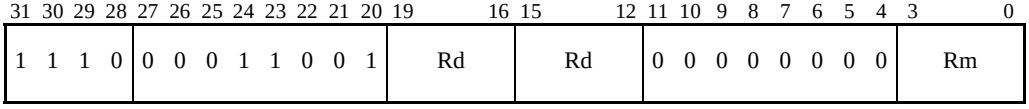
None

Operation

```
Rd = Rd OR Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

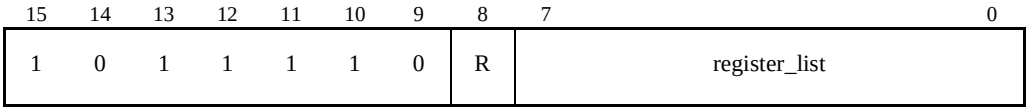
Operation

```
ORRS  <Rd>, <Rd>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.47 POP



The POP (Pop Multiple Registers) instruction loads a subset (or possibly all) of the general-purpose registers R0-R7 and the PC from the stack.

The general-purpose registers loaded can include the PC. If they do, the word loaded for the PC is treated as an address and a branch occurs to that address. In ARM architecture version 5 and above, bit[0] of the loaded value determines whether execution continues after this branch in ARM state or in Thumb state, as though the following instruction had been executed:

```
BX (loaded_value)
```

In T variants of architecture version 4, bit[0] of the loaded value is ignored and execution continues in Thumb state, as though the following instruction had been executed:

```
MOV PC,(loaded_value)
```

Syntax

```
POP  <registers>
```

where:

`<registers>` Is the list of registers, separated by commas and surrounded by { and }. The list is encoded in the register_list field of the instruction, by setting bit[i] to 1 if register Ri is included in the list and to 0 otherwise, for each of i=0 to 7. The R bit (bit[8]) is set to 1 if the PC is in the list and to 0 otherwise.

At least one register must be loaded. If bits[8:0] are all zero, the result is UNPREDICTABLE.

The registers are loaded in sequence, the lowest-numbered register from the lowest memory address (`start_address`), through to the highest-numbered register from the highest memory address (`end_address`). If the PC is specified in the register list (opcode bit[8] is set), the instruction causes a branch to the address (data) loaded into the PC.

The `<start_address>` is the value of the SP.

Subsequent addresses are formed by incrementing the previous address by four. One address is produced for each register that is specified in `<registers>`.

The `end_address` value is four less than the sum of the value of the SP and four times the number of registers specified in `<registers>`.

The SP register is incremented by four times the numbers of registers in `<registers>`.

Copyright © 1996-2000 ARM Limited. All rights reserved.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
start_address = SP
end_address = SP + 4*(R + Number_Of_Set_Bits_In(register_list))
address = start_address

for i = 0 to 7
    if register_list[i] == 1 then
        Ri = Memory[address,4]
        address = address + 4

if R == 1 then
    value = Memory[address,4]
    PC = value AND 0xFFFFFFFE
    if (architecture version 5 or above) then
        T Bit = value[0]
    address = address + 4

assert end_address = address
SP = end_address
```

Usage

The POP instruction is useful for stack operations. A POP instruction with the PC in the register list can be used for an efficient procedure exit, as it restores saved registers, loads the PC with the return address, and updates the stack pointer with a single instruction.

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

CPSR The CPSR is not updated when POP loads the PC.

Alignment POP instructions ignore the least significant two bits of `address`.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception.

 Copyright © 1996-2000 ARM Limited. All rights reserved.

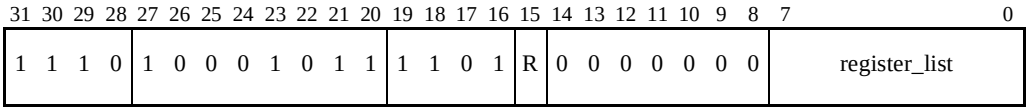
ARM/Thumb state transfers

In ARM architecture 5 and above, if bits[1:0] of a value loaded for R15 are 0b10, the result is UNPREDICTABLE, as branches to non word-aligned addresses are not possible in ARM state.

Time order The time order of the accesses to individual words of memory generated by this instruction is only defined in some circumstances. See *Data accesses to memory-mapped I/O* on page A2-32 for details.

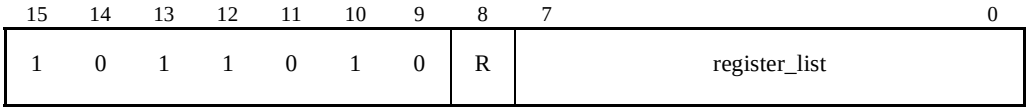
Equivalent ARM syntax and encoding

```
LDMIA  SP!, <registers>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.48 PUSH



The PUSH (Push Multiple Registers) instruction stores a subset (or possibly all) of the general-purpose registers R0-R7 and the LR to the stack.

Syntax

```
PUSH  <registers>
```

where:

`<registers>` Is the list of registers to be stored, separated by commas and surrounded by { and }. The list is encoded in the register_list field of the instruction, by setting bit[i] to 1 if register Ri is included in the list and to 0 otherwise, for each of i=0 to 7. The R bit (bit[8]) is set to 1 if the LR is in the list and to 0 otherwise.

At least one register must be stored. If bits[8:0] are all zero, the result is UNPREDICTABLE.

The registers are stored in sequence, the lowest-numbered register to the lowest memory address (`start_address`), through to the highest-numbered register to the highest memory address (`end_address`)

The `start_address` is the value of the SP minus 4 times the number of registers to be stored.

Subsequent addresses are formed by incrementing the previous address by four. One address is produced for each register that is specified in `<registers>`.

The `end_address` value is four less than the original value of SP.

The SP register is decremented by four times the numbers of registers in `<registers>`.

Architecture version

All T variants

Exceptions

Data Abort

Copyright © 1996-2000 ARM Limited. All rights reserved.

Operation

```
start_address = SP - 4*(R + Number_Of_Set_Bits_In(register_list))
end_address = SP - 4
address = start_address
for i = 0 to 7
    if register_list[i] == 1
        Memory[address,4] = Ri
        address = address + 4
if R == 1
    Memory[address,4] = LR
    address = address + 4
assert end_address == address - 4
SP = SP - 4*(R + Number_Of_Set_Bits_In(register_list))
```

Usage

PUSH is useful for stack operations. A PUSH instruction with the LR in the register list can be used for an efficient procedure entry, as it saves registers (including the return address) on the stack and updates the stack pointer with a single instruction. A matching POP instruction can be used later to return from the procedure.

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

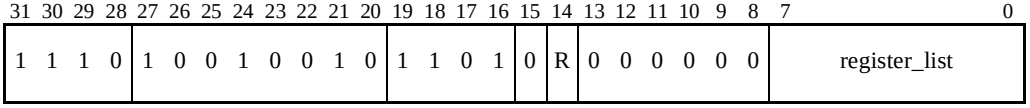
Alignment PUSH instructions ignore the least significant two bits of `address`.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception.

Time order The time order of the accesses to individual words of memory generated by this instruction is only defined in some circumstances. See *Data accesses to memory-mapped I/O* on page A2-32 for details.

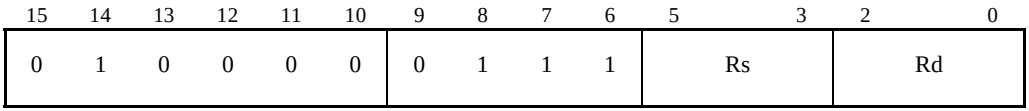
Equivalent ARM syntax and encoding

```
STMDB  SP!, <registers>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.49 ROR



The ROR (Rotate Right Register) instruction is used to provide the value of a register rotated by a variable value (from another register). The bits that are rotated off the right end are inserted into the vacated bit positions on the left. The condition code flags are updated, based on the result.

Syntax

```
ROR  <Rd>, <Rs>
```

where:

<Rd> Contains the value to be rotated, and is also the destination register for the operation.

<Rs> Is the register containing the rotation applied to the value of <Rd>. The value of the rotation is stored in the least significant byte.

Architecture version

All T variants

Exceptions

None

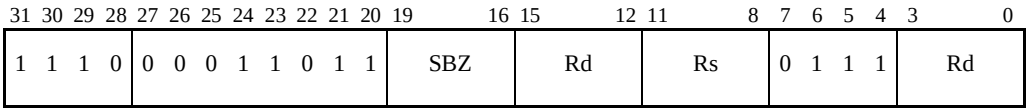
Operation

```
if Rs[7:0] == 0 then
    C Flag = unaffected
    Rd = unaffected
else if Rs[4:0] == 0 then
    C Flag = Rd[31]
    Rd = unaffected
else /* Rs[4:0] > 0 */
    C Flag = Rd[Rs[4:0] - 1]
    Rd = Rd Rotate_Right Rs[4:0]
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
V Flag = unaffected
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

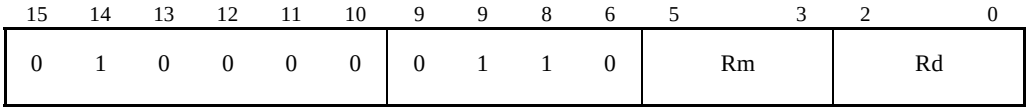
Equivalent ARM syntax and encoding

```
MOVS  <Rd>, <Rd>, ROR <Rs>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.50 SBC



The SBC (Subtract with Carry) instruction can be used to synthesize multi-word subtraction. It subtracts the value of register <Rm> and the value of NOT(Carry Flag) from the value of register <Rd>. The condition code flags are updated, based on the result.

Syntax

```
SBC  <Rd>, <Rm>
```

where:

<Rd> Contains the first operand for the subtraction, and is also the destination register for the operation.

<Rm> Contains the value to be subtracted from <Rd>.

Architecture version

All T variants

Exceptions

None

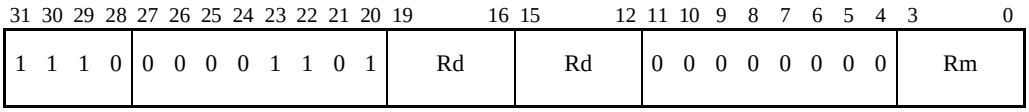
Operation

```
Rd = Rd - Rm - NOT(C Flag)
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rd - Rm - NOT(C Flag))
V Flag = OverflowFrom(Rd - Rm - NOT(C Flag))
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

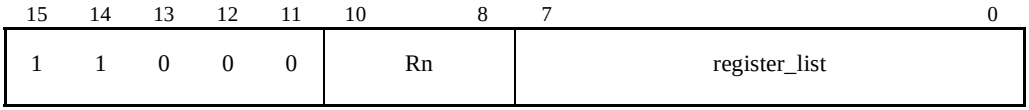
Equivalent ARM syntax and encoding

```
SBCS  <Rd>, <Rd>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.51 STMIA



The STMIA (Store Multiple Increment After) instruction stores a non-empty subset, or possibly all, of the general-purpose registers to sequential memory locations.

Syntax

```
STMIA  <Rn>!, <registers>
```

where:

`<Rn>` Is the register containing the start address for the instruction.

`!` Causes base register writeback, and is not optional.

`<registers>` Is a list of registers to be stored, separated by commas and surrounded by { and }. The list is encoded in the register_list field of the instruction, by setting bit[i] to 1 if register Ri is included in the list and to 0 otherwise, for each of i=0 to 7.

At least one register must be stored. If bits[7:0] are all zero, the result is UNPREDICTABLE.

The registers are stored in sequence, the lowest-numbered register to the lowest memory address (`start_address`), through to the highest-numbered register to the highest memory address (`end_address`).

The `start_address` is the value of the base register `<Rn>`. Subsequent addresses are formed by incrementing the previous address by four. One address is produced for each register that is specified in `<registers>`.

The `end_address` value is four less than the sum of the value of the base register and four times the number of registers specified in `<registers>`.

Finally, the base register `<Rn>` is incremented by 4 times the numbers of registers in `<registers>`.

Architecture version

All T variants

Exceptions

Data Abort

Copyright © 1996-2000 ARM Limited. All rights reserved.

Operation

```
start_address = Rn
end_address = Rn + (Number_Of_Set_Bits_In(register_list) * 4) - 4
address = start_address
for i = 0 to 7
    if register_list[i] == 1
        Memory[address,4] = Ri
        address = address + 4
assert end_address == address - 4
Rn = Rn + (Number_Of_Set_Bits_In(register_list) * 4)
```

Usage

`STMIA` is useful as a block store instruction. Combined with `LDMIA` (Load Multiple), it allows efficient block copy.

Notes

Operand restrictions

If `<Rn>` is specified in `<registers>` and writeback is specified:

- If `<Rn>` is the lowest-numbered register specified in `<registers>`, the original value of `<Rn>` is stored.
- Otherwise, the stored value of `<Rn>` is UNPREDICTABLE.

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

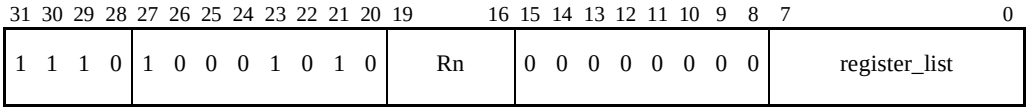
Alignment Store Multiple instructions ignore the least significant two bits of `address`.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception.

Time order The time order of the accesses to individual words of memory generated by this instruction is only defined in some circumstances. See *Data accesses to memory-mapped I/O* on page A2-32 for details.

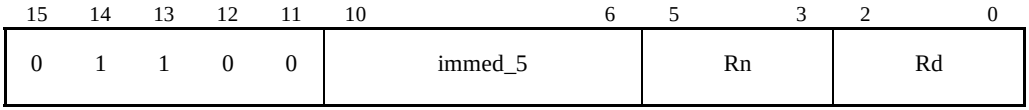
Equivalent ARM syntax and encoding

```
STMIA  <Rn>!, <registers>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.52 STR (1)



This form of the STR (Store Register) instruction allows 32-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for accessing structure (record) fields. With an offset of zero, the address produced is the unaltered value of the base register <Rn>.

Syntax

```
STR  <Rd>, [<Rn>, #<immed_5> * 4]
```

where:

<Rd> Is the register that contains the word to be stored to memory.

<Rn> Is the register containing the base address for the instruction.

<immed_5> Is a 5-bit value that is multiplied by 4 and added to the value of <Rn> to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + (immed_5 * 4)
if address[1:0] == 0b00
    Memory[address,4] = Rd
else
    Memory[address,4] = UNPREDICTABLE
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

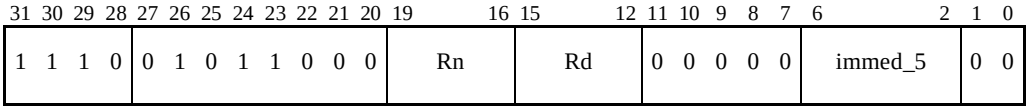
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not word-aligned and no data abort occurs, the value written to memory is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

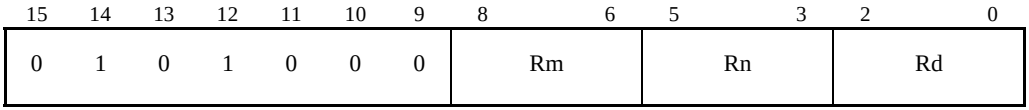
Equivalent ARM syntax and encoding

```
STR  <Rd>, [<Rn>, #<immed_5> * 4]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.53 STR (2)



This form of STR allows 32-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for pointer + large offset arithmetic, and for accessing a single element of an array.

Syntax

```
STR  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the register that contains the word to be stored to memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register containing the second value used in forming the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + Rm
if address[1:0] == 0b00
    Memory[address,4] = Rd
else
    Memory[address,4] = UNPREDICTABLE
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

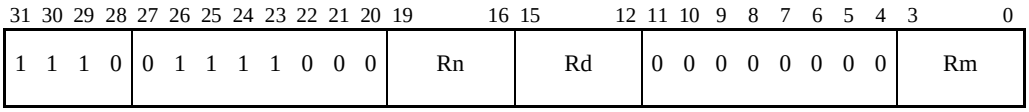
Alignment If the memory address is not word-aligned and no data abort occurs, the value written to memory is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

 Copyright © 1996-2000 ARM Limited. All rights reserved.

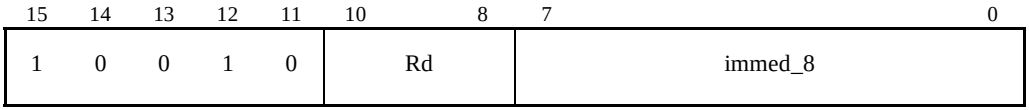
Equivalent ARM syntax and encoding

```
STR  <Rd>, [<Rn>, <Rm>]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.54 STR (3)



This form of STR allows 32-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for accessing stack data. In this case, STR stores a word from register <Rd> to memory.

Syntax

```
STR  <Rd>, [SP, #<immed_8> * 4]
```

where:

<Rd> Is the register that contains the word to be stored to memory.

SP Is the stack pointer. Its value is used to calculate the memory address.

<immed_8> Is an 8-bit value that is multiplied by 4 and added to the value of the SP to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = SP + (immed_8 * 4)
if address[1:0] == 0b00
    Memory[address,4] = Rd
else
    Memory[address,4] = UNPREDICTABLE
```

 Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

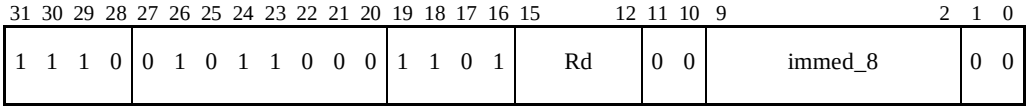
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not word-aligned and no data abort occurs, the value written to memory is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*), and alignment checking is enabled, an address with bits[1:0] != 0b00 causes an alignment exception (a type of data abort).

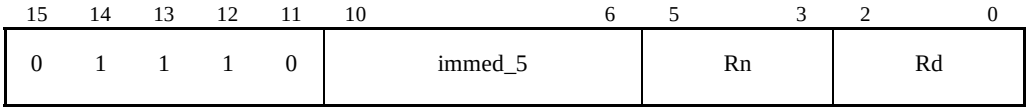
Equivalent ARM syntax and encoding

```
STR  <Rd>, [SP, #<immed_8> * 4]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.55 STRB (1)



This form of the STRB (Store Register Byte) instruction allows 8-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for accessing structure (record) fields.

With an offset of zero, the address produced is the unaltered value of the base register <Rn>.

Syntax

```
STRB  <Rd>, [<Rn>, #<immed_5>]
```

where:

<Rd> Is the register whose least significant byte is stored to memory.

<Rn> Is the register containing the base address for the instruction.

<immed_5> Is a 5-bit immediate value that is added to the value of <Rn> to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

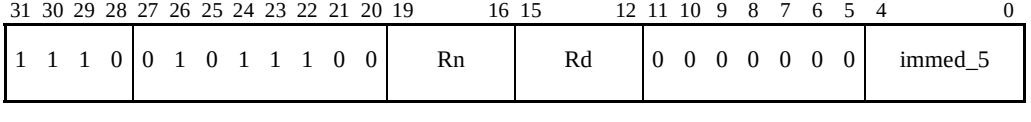
```
address = Rn + immed_5
Memory[address,1] = Rd[7:0]
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

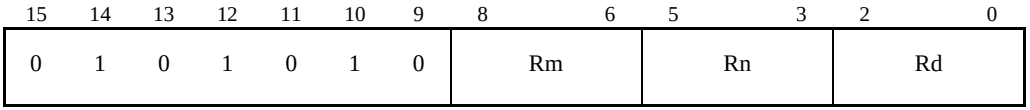
Equivalent ARM syntax and encoding

```
STRB  <Rd>, [<Rn>, #<immed_5>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.56 STRB (2)



This form of STRB allows 8-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for pointer + large offset arithmetic, and for accessing a single element of an array.

Syntax

```
STRB  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the register whose least significant byte is stored to memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register whose value is added to `<Rn>` to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

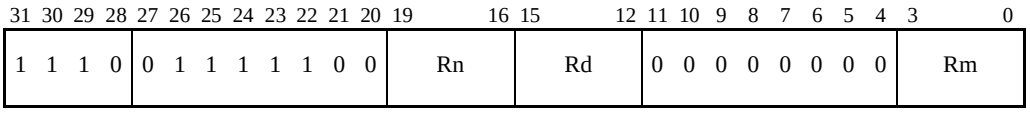
```
address = Rn + Rm
Memory[address,1] = Rd[7:0]
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

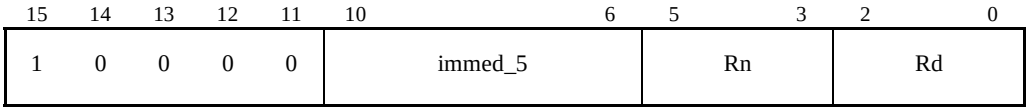
Equivalent ARM syntax and encoding

```
STRB  <Rd>, [<Rn>, <Rm>]
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.57 STRH (1)



This form of the STRH (Store Register Halfword) instruction allows 16-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for accessing structure (record) fields. With an offset of zero, the address produced is the unaltered value of the base register <Rn>.

Syntax

```
STRH  <Rd>, [<Rn>, #<immed_5> * 2]
```

where:

<Rd> Is the register whose least significant halfword is stored to memory.

<Rn> Is the register containing the base address for the instruction.

<immed_5> Is a 5-bit immediate value that is multiplied by two and added to the value of <Rn> to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + (immed_5 * 2)
if address[1:0] == 0
    Memory[address,2] = Rd[15:0]
else
    Memory[address,2] = UNPREDICTABLE
```


Copyright © 1996-2000 ARM Limited. All rights reserved.

Notes

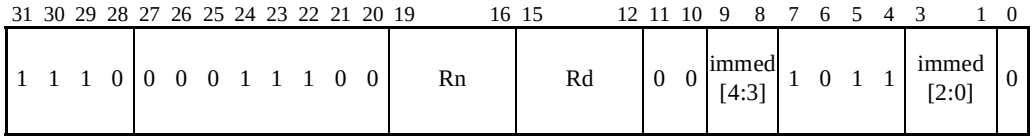
Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

Alignment If the memory address is not halfword-aligned and no data abort occurs, the value written to memory is UNPREDICTABLE.

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bit[0] != 0 causes an alignment exception (a type of data abort).

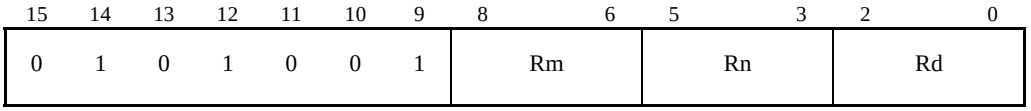
Equivalent ARM syntax and encoding

```
STRH  <Rd>, [<Rn>, #<immed_5> * 2]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.58 STRH (2)



This form of STRH allows 16-bit data from a general-purpose register to be stored to memory. The addressing mode is useful for pointer + large offset arithmetic and for accessing a single element of an array.

Syntax

```
STRH  <Rd>, [<Rn>, <Rm>]
```

where:

`<Rd>` Is the register whose least significant halfword is stored to memory.

`<Rn>` Is the register containing the first value used in forming the memory address.

`<Rm>` Is the register whose value is added to `<Rn>` to form the memory address.

Architecture version

All T variants

Exceptions

Data Abort

Operation

```
address = Rn + Rm
if address[1:0] == 0
    Memory[address,2] = Rd[15:0]
else
    Memory[address,2] = UNPREDICTABLE
```

Notes

Data abort For details of the effects of the instruction if a data abort occurs, see *Effects of data-aborted instructions* on page A2-17.

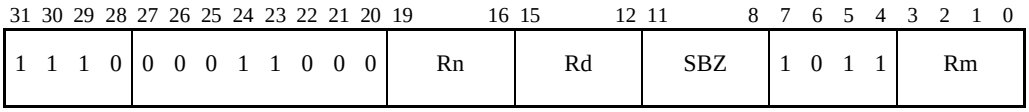
Alignment If the memory address is not halfword-aligned and no data abort occurs, the value written to memory is UNPREDICTABLE

If an implementation includes a System Control coprocessor (see Chapter B2 *The System Control Coprocessor*) and alignment checking is enabled, an address with bit[0] != 0 causes an alignment exception (a type of data abort).

 Copyright © 1996-2000 ARM Limited. All rights reserved.

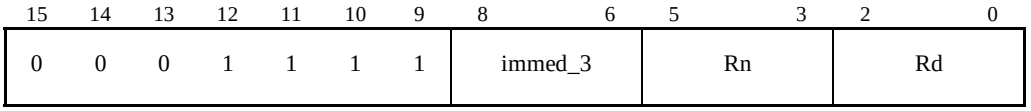
Equivalent ARM syntax and encoding

```
STRH  <Rd>, [<Rn>, <Rm>]
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.59 SUB (1)



This form of the SUB (Subtract) instruction subtracts a small constant value from the value of a register and stores the result in a second register. The condition code flags are updated, based on the result.

Syntax

```
SUB  <Rd>, <Rn>, #<immed_3>
```

where:

`<Rd>` Is the destination register for the operation.

`<Rn>` Is the register containing the first operand for the subtraction.

`<immed_3>` Is a 3-bit immediate value (values 0 to 7) that is subtracted from `<Rn>`.

Architecture version

All T variants

Exceptions

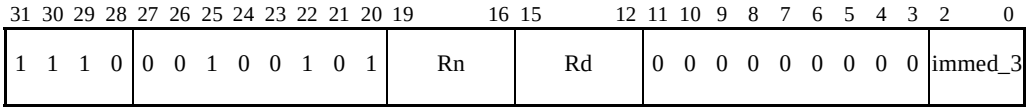
None

Operation

```
Rd = Rn - immed_3
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn - immed_3)
V Flag = OverflowFrom(Rn - immed_3)
```

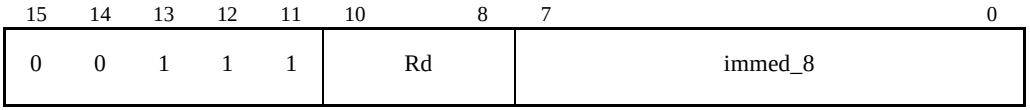
Equivalent ARM syntax and encoding

```
SUBS  <Rd>, <Rn>, #<immed_3>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.60 SUB (2)



This form of SUB subtracts a large immediate value from the value of a register and stores the result back in the same register. The condition code flags are updated, based on the result.

Syntax

```
SUB  <Rd>, #<immed_8>
```

where:

`<Rd>` Is the register containing the first operand for the subtraction, and is also the destination register for the operation.

`<immed_8>` Is an 8-bit immediate value (values 0 to 255) that is subtracted from `<Rd>`.

Architecture version

All T variants

Exceptions

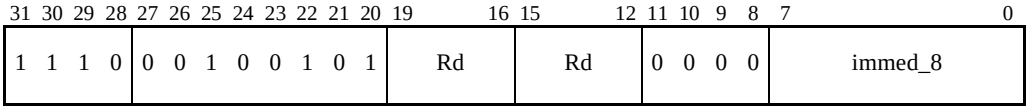
None

Operation

```
Rd = Rd - immed_8
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rd - immed_8)
V Flag = OverflowFrom(Rd - immed_8)
```

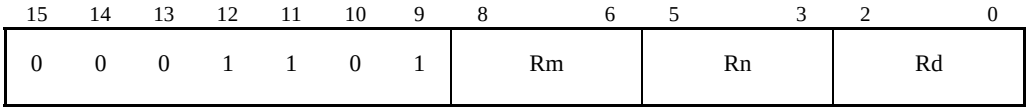
Equivalent ARM syntax and encoding

```
SUBS  <Rd, <Rd>, #<immed_8>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.61 SUB (3)



This form of SUB subtracts the value of one register from the value of a second register and stores the result in a third register. The condition code flags are updated, based on the result.

Syntax

```
SUB  <Rd>, <Rn>, <Rm>
```

where:

`<Rd>` Is the destination register for the operation.

`<Rn>` Is the register containing the first operand for the subtraction.

`<Rm>` Is the register whose value is subtracted from `<Rn>`.

Architecture version

All T variants

Exceptions

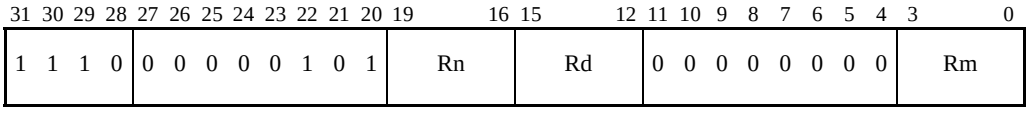
None

Operation

```
Rd = Rn - Rm
N Flag = Rd[31]
Z Flag = if Rd == 0 then 1 else 0
C Flag = NOT BorrowFrom(Rn - Rm)
V Flag = OverflowFrom(Rn - Rm)
```

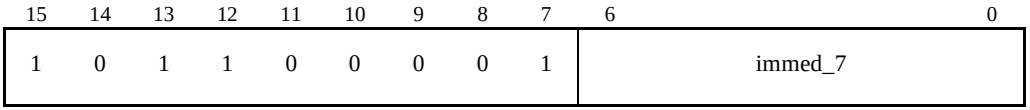
Equivalent ARM syntax and encoding

```
SUBS  <Rd>, <Rn>, <Rm>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.62 SUB (4)



This form of SUB decrements the SP by four times a 7-bit immediate (that is, by a multiple of 4 in the range 0 to 508). The condition codes are not affected.

Syntax

```
SUB  SP, #<immed_7> * 4
```

where:

SP — Indicates the stack pointer. The result of the operation is also stored in the SP.

<immed_7> — Is a 7-bit immediate value that is multiplied by 4 and then subtracted from the value of the stack pointer.

Architecture version

All T variants

Exceptions

None

Operation

```
SP = SP - (immed_7 << 2)
```

Usage

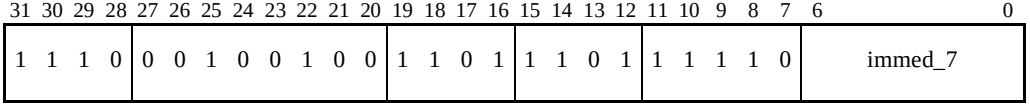
For the Full Descending stack which the Thumb instruction set is designed to use, decrementing the SP is used to allocate extra memory variables on the top of the stack.

Notes

Alternative syntax This instruction can also be written as `SUB SP, SP, #<immed_7> * 4`.

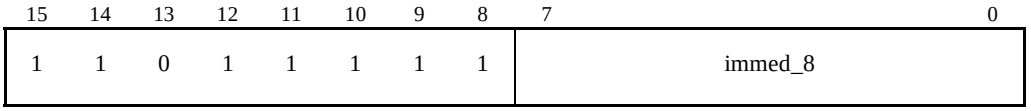
Equivalent ARM syntax and encoding

```
SUB  SP, SP, #<immed_7> * 4
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.63 SWI



The `SWI` (Software Interrupt) instruction is used as an operating system service call. A SWI exception is generated, which is handled by an operating system to provide the requested service. See *Exceptions* on page A2-13.

Syntax

```
SWI  <immed_8>
```

where:

`<immed_8>` Is an 8-bit immediate value that is put into bits[7:0] of the instruction. This value is ignored by the processor, but can be used by an operating system's SWI exception handler to determine which operating system service is being requested.

Architecture version

All T variants

Exceptions

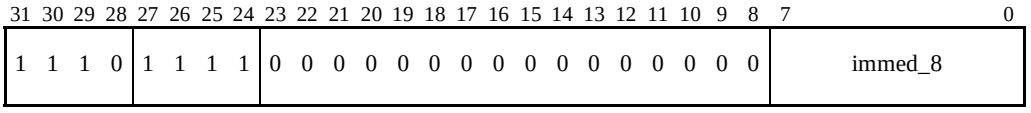
Software Interrupt

Operation

```
R14_svc   = address of next instruction after the SWI instruction
SPSR_svc  = CPSR
CPSR[4:0] = 0b10011              /* Enter Supervisor mode */
CPSR[5]   = 0                    /* Execute in ARM state */
/* CPSR[6] is unchanged */
CPSR[7]   = 1                    /* Disable normal interrupts */
if high vectors configured then
    PC    = 0xFFFF0008
else
    PC    = 0x00000008
```

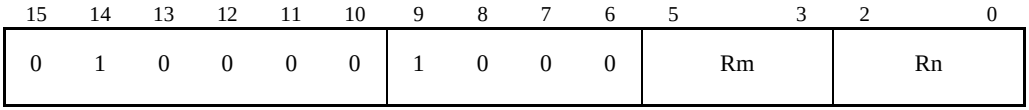
Equivalent ARM syntax and encoding

```
SWI  <immed_8>
```



 Copyright © 1996-2000 ARM Limited. All rights reserved.

7.1.64 TST



The TST (Test) instruction is used to determine whether a particular subset of bits in a register includes at least one set bit. A very common use for TST is to test whether a single bit is set or clear. The condition code flags are updated, based on the result.

Syntax

```
TST  <Rn>, <Rm>
```

where:

`<Rn>` Is the register containing the first operand for the instruction.

`<Rm>` Is the register whose value is logically ANDed with the value of `<Rn>`.

Architecture version

All T variants

Exceptions

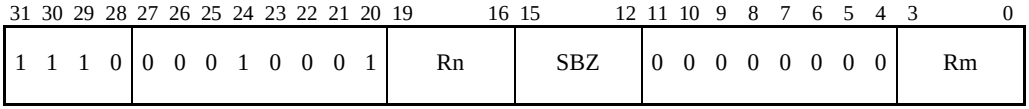
None

Operation

```
alu_out = Rn AND Rm
N Flag = alu_out[31]
Z Flag = if alu_out == 0 then 1 else 0
C Flag = unaffected
V Flag = unaffected
```

Equivalent ARM syntax and encoding

```
TST  <Rn>, <Rm>
```



Copyright © 1996-2000 ARM Limited. All rights reserved.

7.2 Thumb instructions and architecture versions

Table 7-1 shows which Thumb instructions are present in each current ARM architecture version that supports Thumb.

Table 7-1 Thumb instructions by architecture

Instruction	v4T, v4TxM	v5T, v5TxM
ADC	Yes	Yes
ADD (all forms)	Yes	Yes
AND	Yes	Yes
ASR (both forms)	Yes	Yes
B (both forms)	Yes	Yes
BIC	Yes	Yes
BKPT	No	Yes
BL	Yes	Yes
BLX (both forms)	No	Yes
BX	Yes	Yes
CMN	Yes	Yes
CMP (all forms)	Yes	Yes
EOR	Yes	Yes
LDMIA	Yes	Yes
LDR (all forms)	Yes	Yes
LDRB (both forms)	Yes	Yes
LDRH (both forms)	Yes	Yes
LDRSB	Yes	Yes
LDRSH	Yes	Yes
LSL (both forms)	Yes	Yes
LSR (both forms)	Yes	Yes
MOV (all forms)	Yes	Yes

Copyright © 1996-2000 ARM Limited. All rights reserved.

Table 7-1 Thumb instructions by architecture *(Continued)*

Instruction	v4T, v4TxM	v5T, v5TxM
MUL	Yes	Yes
MVN	Yes	Yes
NEG	Yes	Yes
ORR	Yes	Yes
POP	Yes	Yes
PUSH	Yes	Yes
ROR	Yes	Yes
SBC	Yes	Yes
STMIA	Yes	Yes
STR (all forms)	Yes	Yes
STRB (both forms)	Yes	Yes
STRH (both forms)	Yes	Yes
SUB (all forms)	Yes	Yes
SWI	Yes	Yes
TST	Yes	Yes

Copyright © 1996-2000 ARM Limited. All rights reserved.

Copyright © 1996-2000 ARM Limited. All rights reserved.